MARS: Episode-Scoped GPU Retrieval for Real-Time Embodied AI

with Native Temporal Decay and Cross-Modal Bridges

Antonello Fratepietro
Independent Researcher
https://github.com/antonellof/MARS

April 2026

Abstract

Real-time AI systems — autonomous vehicles, humanoid robots, AR/VR headsets — issue memory retrieval queries at sensor rate (30–1000 Hz) under tight latency budgets. Existing GPU vector search libraries (FAISS, cuVS) expose a single contract: rank by cosine similarity over the entire indexed corpus. Embodied workloads typically know more than that at query time: an active track id, a dialogue session id, a current sub-task, or a recency window over which a memory is still operative. Encoding this knowledge as host-side filtering over the output of a global ANN sweep wastes GPU work and, as we show, breaks recall at scale on tiny multimodal clusters.

We present MARS (**M**emory for **A**utonomous **R**eal-time **S**ystems), a GPU-resident memory store whose retrieval API exposes three orthogonal ranking primitives as first-class kernel-level controls: *(i)* per-query `RetrievalScope` that restricts the cosine sweep to the members of a named episode, *(ii)* exponential temporal decay folded into the score *before* top-K selection, and *(iii)* a bounded warp-cooperative BFS over a Neural Shortcut Network (NSN) with cross-modal bridges. The data path is four GPU kernels — cuBLAS `Sgemv`, lightweight decay, CUB radix sort, BFS — operating entirely on device-resident state with no per-query allocation.

Key results (single A100 SXM4, same hardware and probe set as the baselines unless noted; see hardware index in Table 5):

- **Episode-scoped retrieval** delivers **197** μ**s** $p99$ **at** $N{=}1$ **M** on a synthetic embodied multimodal benchmark (10-node cross-modal episodes of one IMAGE, three AUDIO and six TEXT vectors; image query $\rightarrow$ same-episode text+audio; full setup in §7.12), while keeping perfect cross-modal hit@15. This is $33\times$ faster than the only other system that retains hit@15$=1.0$ at that scale (FAISS-Flat-GPU, 6.64 ms) and the only configuration that stays under the 1 ms AV deadline at $N{=}10^6$.
- Cosine-only GPU ANN baselines (FAISS-IVF, cuVS CAGRA) lose hit@15 on this same benchmark as N grows ($\leq$ 0.4 at $N{=}10^5$, 0 at $N{=}10^6$). The dominant signal at scale is not ANN parameters but *whether* the kernel sees episode membership.
- Native temporal decay matches a two-stage FAISS-with-post-hoc filter pipeline at the same TP@10$=0.910$ in AV perception (0.26 ms vs. 0.25 ms $p99$). The contribution here is API consolidation, not raw speed.
- Wall-clock latency on the global path is 1.4–2.2$\times$ FAISS Flat at $N{\leq}10^4$ (a fixed $\sim$0.15 ms overhead for the decay and BFS stages), narrowing to parity at $N{=}50$K. The optimized cuBLAS+CUB path scales to 13 M memories at 29 ms $p99$ on a single 40 GB A100.
- All four demonstrator empirical $p99$ deadlines met: AV 0.87 ms (budget 1 ms), robot 0.76 ms (budget 1 ms), AR/VR 1.56 ms (budget 5 ms), voice 0.88 ms (budget 20 ms).

MARS fills a specific gap in the GPU retrieval landscape: between fast-but-stateless circular buffers and capable-but-slow vector databases, by treating *ranking* (scope, time, modality) as a kernel-level primitive rather than as host-side post-processing.

Keywords: real-time systems, episode-scoped retrieval, temporal decay, GPU memory, autonomous driving, multimodal retrieval, CUDA, streaming insertion.

1 Introduction

1.1 The Problem: Embodied Memory Has Structure That ANN Doesn't See

An autonomous vehicle's perception stack runs at 60 Hz. A child's ball rolls into the road from behind a parked van. Vision alone sees only the ball. But 600 ms earlier, the vehicle's microphones captured children's voices from that same direction — a memory that, if retrievable now, raises the prior that a child may follow the ball into the street. The query isn't "find the most visually similar memory"; it's "find any recent sensor evidence, across any modality, relevant to *this current track*." A voice sample from last week's drive through a schoolyard has high audio similarity but is useless. The useful memory is 600 ms old, from a different modality than the query, and low in cosine similarity compared to countless irrelevant-but-similar alternatives in the corpus.

We use this scene as the paper's running example and refer to it throughout as the *kids-ball benchmark*: a synthetic multimodal corpus of 10-node cross-modal episodes (one IMAGE, three AUDIO, six TEXT vectors per episode; within-episode cosine > 0.85, across-episode $\sim$0.2–0.4 in a shared 768-D space) queried with an IMAGE probe whose ground-truth answer is the same-episode TEXT+AUDIO neighbours. The full generator and probe contract are described in §7.12; we surface the name here so every later headline number can be read against this single, fixed contract.

Two things are true at once for the embodied perception stack and both are missed by a pure cosine-only ANN sweep:

1. **The application typically knows what to focus on.** The perception stack already maintains a notion of *episode*: a tracked object id, a dialogue session id, an AR room id, a robot sub-task. When the application can supply this id at query time, the right answer almost always lives inside the episode — the cross-modal voice memory above belongs to "this track" even when it does not look like it under cosine.

2. **Temporal proximity dominates relevance.** For a tracking query, a six-month-old detection of the same object is functionally noise. Recency is part of the ranking, not a post-hoc filter.

Existing GPU-accelerated vector search libraries — FAISS [10], cuVS CAGRA [23], and similar systems — expose a single contract: find the K vectors with highest cosine similarity to a query, over the entire indexed corpus. They treat all indexed vectors as equally valid regardless of age, modality, or episode membership. For static corpora — document retrieval, recommendation, image search — this is the correct formulation. For streaming sensor data with episodic structure, it is the wrong abstraction.

Embodied applications today work around the abstraction mismatch with host-side post-processing: run a global cosine sweep, then filter the top-K' to discard stale or wrong-episode results. This works when $K' \gg K$ and when the right answers happen to rank in the global top-K', but as we show in Section 7.12 it *breaks recall at scale*: at $N{=}10^6$ with 10-node multimodal episodes, both FAISS-IVF and cuVS CAGRA drop to ≤ 0.4 hit@15 — not because their ANN parameters are wrong but because the per-episode TEXT and AUDIO neighbors are buried among $\sim$999 990 distractors. Only an *exhaustive* cosine sweep keeps recall, and exhaustive cosine at $N{=}10^6$ costs more than the 1 ms AV deadline on a single A100.

1.2 MARS: Ranking as a Kernel-Level Primitive

MARS treats the three signals embodied perception cares about — *scope*, *time*, and *cross-modal connectivity* — as first-class controls of the retrieval kernel rather than as host-side post-processing.

Scope as a kernel parameter. The retrieval API takes a `RetrievalScope` that names the episode the application is already tracking. When the scope is `Global`, the kernel runs the conventional cosine sweep over V. When the scope is `EpisodeScoped(id)`, Stage 1 is restricted to the CSR member range of that episode and BFS depth is set to zero. The work collapses from $\Theta(N \cdot D)$ to $\Theta(|\text{episode}| \cdot D)$, which on the kids-ball benchmark just defined (Section 7.12) lands at **197** μ**s** $p99$ independent of N across three decades. *This is the empirical centrepiece of the paper.*

Temporal decay as a kernel epilogue. Rather than relying on a post-hoc filter pass, every retrieval applies an exponential time-decayed score *before* top-K selection:

$$s(v, q, t_q) = \langle E(v), q \rangle \cdot \exp\bigl(-\lambda \cdot (t_q - \tau(v))\bigr),$$

where λ is a workload-tunable decay rate. The arithmetic content here is small (one multiplication and one `__expf`); the contribution is API consolidation. We make the cost-of-API-consolidation explicit in Section 6.1: identical temporal precision to a two-stage FAISS+filter pipeline at 0.26 ms vs. 0.25 ms $p99$.

Cross-modal bridges in the graph topology. The NSN graph guarantees that a bounded-hop BFS from any seed reaches every modality (Section 5). The contribution here is *structural*: we make no claim that BFS-discovered nodes are semantically optimal — only that they are structurally reachable on a single GPU pass.

Streaming inserts. New memories become queryable immediately — a CSR append plus an NSN edge insert, no rebuild step. This is comparable in spirit to the recent SIVF and CAGRA `extend()` primitives discussed in Section 9; MARS's contribution is again on the *ranking* side, not on the data-structure side.

1.3 Contributions

1. **Episode-scoped retrieval as a GPU kernel-level primitive** (Section 7.11, 7.12). On the embodied kids-ball multimodal contract with paired probes, episode-scoped MARS delivers **197** μ**s** $p99$ **at** N=1 **M** (13× vs. the MARS global path, 33× vs. FAISS-Flat-GPU, the only baseline that retains `hit@15`=1.0 at that scale) and is the only configuration that stays under the 1 ms AV deadline at N=10^6.

2. **An empirical recall-at-scale failure of cosine-only GPU ANN on multimodal episodes** (Section 7.12). FAISS-IVF and cuVS CAGRA drop to $\leq$ 0.4 `hit@15` at N=10^5 and 0 at N=10^6 on the kids-ball contract; even widening CAGRA's beam to `search_k`=512 does not recover recall. This is the empirical justification for encoding episode membership in the kernel API rather than relying on host-side filtering of an ANN top-K'.

3. **A three-way latency comparison** for native temporal decay (Section 6) — FAISS cosine-only, FAISS+post-hoc-filter, and MARS with kernel-fused decay — showing that MARS matches FAISS+filter at identical TP@10 (0.910) and a comparable per-query latency (0.26 ms vs. 0.25 ms $p99$). The contribution is consolidating the filter into the kernel API, not raw speedup.

4. **A GPU-resident retrieval pipeline with first-class scope, time, and cross-modal controls** (Section 4). Four kernels (cuBLAS `Sgemv`, decay epilogue, CUB radix sort, warp-cooperative BFS) operate on device-resident state with zero per-query allocation, sharing the same `QueryContext` between the global and episode-scoped contracts.

5. **The Multimodal Neural Shortcut Network** (Section 5) with cross-modal bridges providing bounded-hop structural reachability across modalities. We are careful to label the formal property as *structural*, not semantic; semantic relevance of BFS-discovered nodes depends on encoder alignment (Section 10).

6. **Four deadline-aware demonstrators** (Section 7.4) — AV (60 Hz, 1 ms), robot (1 kHz, 1 ms), AR/VR (90 Hz, 5 ms), voice (30 Hz, 20 ms) — all passing $p99$ budgets on datacenter and consumer GPUs over 30 s sustained runs.

7. **An honest ablation study** (Section 7.6) showing that within-modality recall at $N \leq 50$K is driven entirely by brute-force cosine + temporal decay — the NSN graph contributes structural cross-modal reachability, not within-modality recall. We also report a cautionary methodological negative result on a tunable score-boost multiplier that disappears once probes are paired across configurations.

2 Problem Formulation

This section formalizes the multimodal memory retrieval problem and the temporal and deadline constraints that distinguish this work from conventional approximate nearest-neighbor search.

Definition 1 (Multimodal Memory Corpus)**.** *A multimodal memory corpus is a tuple $\mathcal{C} = (V, E, \phi, \mu, \tau)$ where:*

- $V = \{v_1, \ldots, v_N\}$ *is a set of N memory nodes;*
- $E : V \to \mathbb{R}^D$ *maps each node to an L2-normalized embedding, $\|E(v)\|_2 = 1$;*
- $\mu : V \to \{0, 1, \ldots, M-1\}$ *assigns each node a modality tag (text=0, audio=1, image=2);*
- $\tau : V \to \mathbb{R}_{\geq 0}$ *assigns each node a timestamp;*
- ϕ *is the NSN graph topology over V.*

Definition 2 (Temporally-Decayed Multimodal Retrieval)**.** *Given a query embedding $q \in \mathbb{R}^D$ with $\|q\|_2 = 1$, query timestamp t_q, decay rate λ, and result size K, the retrieval problem is to find:*

$$\text{top-}K_{v \in V}\ s(v, q, t_q)$$

where the time-decayed similarity score is:

$$s(v, q, t_q) = \langle E(v), q \rangle \cdot \exp\big(-\lambda \cdot (t_q - \tau(v))\big).$$

Definition 3 (Temporal Relevance)**.** *For a retrieval result set $R = \{v_1, \ldots, v_K\}$ over a corpus with object identities,* Temporal Precision@K *is the fraction of results whose object identity matches a ground-truth active object* and *whose timestamp falls within the current tracking window $[t_q - w, t_q]$:*

$$\text{TP@}K = \frac{|\{v \in R : \text{active}(v) \wedge \tau(v) \geq t_q - w\}|}{K}.$$

Definition 4 (Streaming Freshness)**.** *A retrieval system has* streaming freshness *if a memory inserted at time t_i is queryable at time $t_i + \epsilon$ for arbitrarily small $\epsilon > 0$, without requiring an explicit index rebuild. Systems without streaming freshness have a* staleness window Δ *during which newly inserted memories are invisible to queries.*

Definition 5 (Deadline-Constrained Retrieval)**.** *A retrieval system satisfies a deadline constraint* (r, B) *if for a sensor rate* r *Hz and latency budget* B *ms, the* $p99$ *wall-clock latency over a sustained run satisfies* $L_{p99} \leq B$.

Definition 6 (Cross-Modal Retrieval)**.** *A retrieval over corpus* $\mathcal{C}$ *is* cross-modal *if the result set contains nodes from at least two distinct modalities:* $|\{\mu(v) : v \in \text{top-}K\}| \geq 2$.

The central design challenge is to solve the retrieval problem of Definition 2 subject to the deadline constraint of Definition 5, with streaming freshness (Definition 4) and cross-modal support (Definition 6). No existing GPU vector search system addresses all four requirements simultaneously.

3 Target Workloads

Table 1 enumerates nine representative workloads spanning safety-critical (AV, robotics, surgical, drones), perception-critical (AR/VR, voice), and UX-critical (games, IoT) tiers. All share four properties: sensor-rate queries, tight latency budgets, native multimodality, and bounded working sets that fit in GPU HBM. Production systems in all tiers currently use hand-rolled C++ circular buffers, each re-implementing similar indexing logic without cross-modal retrieval, temporal awareness, or semantic quality.

Table 1: Representative target workloads. Working set is the number of simultaneously-indexed memories at steady state.

Workload	Rate (Hz)	Budget	Working set	Modalities
AV perception	60	1 ms	2,000–10,000	vision + radar + text
Humanoid control	1000	1 ms	500–6,000	proprio + vision + audio
Surgical robotics	60	100 ms	1,000–50,000	vision + prior scans + text
Drone tracking	100	5 ms	500–2,000	vision + radar
AR/VR SLAM	90	5 ms	5,000–30,000	vision + depth + IMU
Voice agent	30	20 ms	500–3,000	text + audio + vision
Broadcast replay	30	500 ms	10,000–100,000	video + audio + stats
Game NPC	60	16 ms	100–5,000	text + state
IoT predictive	100	10 ms	1,000–10,000	vibration + thermal + audio

4 System Architecture

4.1 Overview

Figure 1 shows the MARS retrieval pipeline. Incoming sensor data flows through modality-specific encoders — E5 [34] for text, CLIP [26] or SigLIP for images, CLAP [38] for audio — that project each modality into a shared 768-D embedding space. Embeddings, modality tags, and timestamps are uploaded into GPU memory and form the nodes of a multimodal Neural Shortcut Network. At query time, the query vector enters the GPU and passes through four kernels operating entirely on device-resident data.

4.2 Data Layout

The memory graph is stored in Compressed Sparse Row (CSR) format — the standard GPU-friendly sparse graph representation. Five device arrays hold the complete state:
For a representative AV working set of N=10,000 at D=768 with average degree 12, the total footprint is approximately 31 MB — well under any modern GPU's HBM.

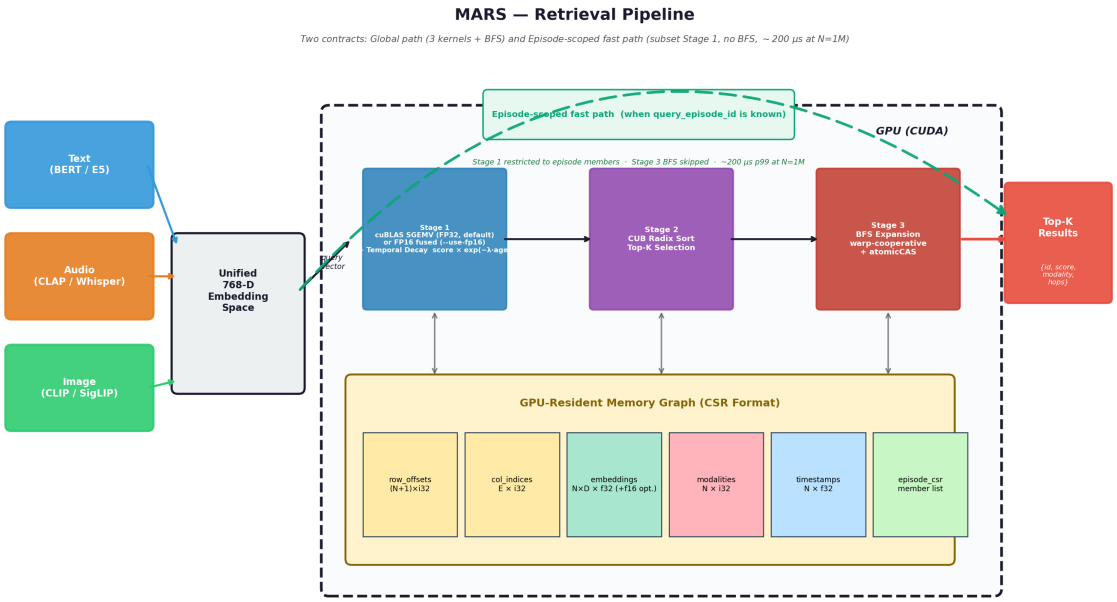

Figure 1: MARS retrieval pipeline. Sensor encoders (left) feed a shared 768-D embedding space; four GPU kernels orchestrate retrieval over a CSR-format memory graph resident in device memory.

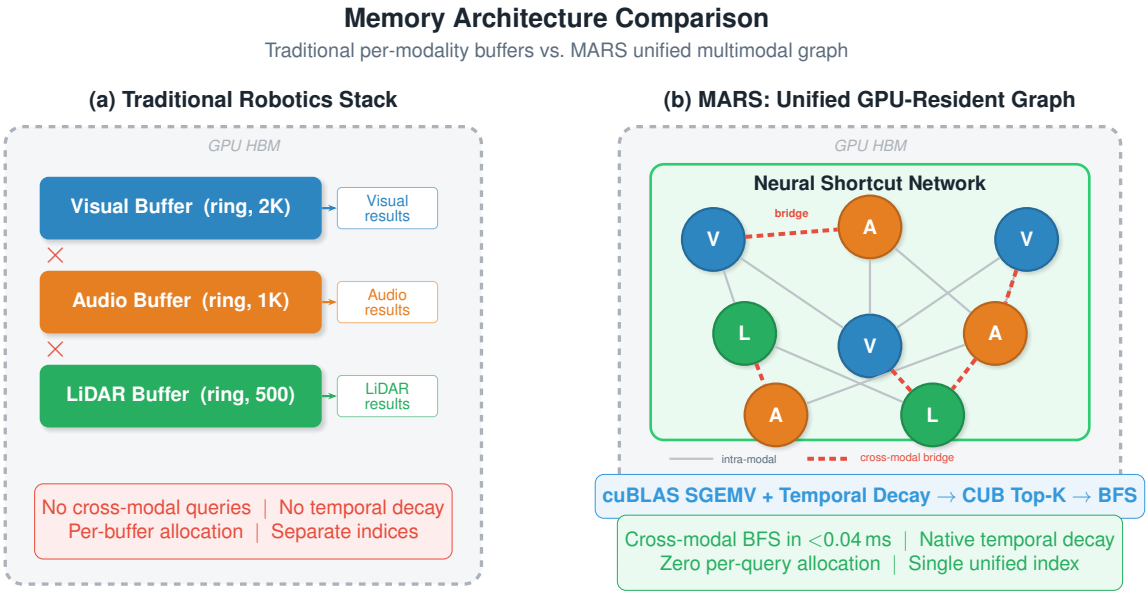

Figure 2: Traditional per-modality circular buffers (a) vs. MARS unified GPU-resident graph with cross-modal NSN bridges (b).

4.3 Four-Kernel Retrieval Pipeline

MARS uses mature GPU library primitives for the compute-heavy stages:

Table 2: Device memory layout for the MARS memory graph.

Array	Size	Type	Purpose
`row_offsets`	$N+1$	int32	Prefix-sum of node degrees
`col_indices`	E	int32	Neighbor node IDs
`embeddings`	$N \times D$	float32	L2-normalized vectors
`modalities`	N	int32	0=text, 1=audio, 2=image
`timestamps`	N	float32	For temporal decay

Stage 1: cuBLAS Similarity. A single `cublasSgemv` call computes the dot product of the query vector against all N embeddings. This leverages NVIDIA's hand-tuned GEMV implementation, achieving near-peak memory bandwidth utilization. Complexity: $\Theta(N \cdot D)$.

Stage 2: Temporal Decay. Applies $\text{score}_i = \text{sim}_i \cdot \exp(-\lambda \cdot (t_q - \tau_i))$ using the `__expf` hardware intrinsic. Implementation note: when using cuBLAS SGEMV (the default, highest-performance path), decay runs as a separate lightweight kernel immediately after SGEMV. When using the custom similarity kernel (available as a fallback), decay is fused into the same kernel launch, eliminating one kernel dispatch and one N-element read/write pass. Both paths produce identical results. The decay constant λ is workload-tunable: higher values produce faster forgetting (AV: $\lambda = 0.5$, 2-second window), lower values preserve older context (voice: $\lambda = 10^{-4}$, 30-minute conversation).

Stage 3: CUB Radix-Sort Top-K. The NVIDIA CUB library's `DeviceRadixSort` sorts scores and indices, followed by truncation to the top K. CUB's radix sort is bandwidth-optimal on GPU and eliminates the custom top-K kernel that dominated latency in earlier pipeline versions ($\sim$0.5 ms in the initial version).

Stage 4: Warp-Cooperative BFS Expansion. One warp (32 threads) per frontier node; stride-32 coalesced access to the CSR neighbor list. A scalar `atomicCAS` on a `uint8_t visited` array provides race-free neighbor claiming. We do *not* use warp-aggregated atomics (Merrill et al. [18], Beamer et al. [2]) because the empirical $K \cdot \bar{d} \leq 200$ frontier per hop in the target regime keeps contention low; in profiling we measured $< 2\%$ of BFS time inside the atomic. For the unbounded-frontier regime (e.g., $K{>}256$ or large h) warp-aggregation would be the correct optimization. The total work per query is $O(K \cdot \bar{d}^h)$ for K seeds, depth h, and maximum degree $\bar{d}$ — bounded by a constant independent of corpus size N (Proposition 8).

atomicCAS contract used in lines 16–17. We rely on the standard CUDA `atomicCAS(addr, cmp, val)` contract: the atomic returns the value previously stored at `addr`, and the swap to `val` succeeds iff that previous value equals `cmp`. The BFS expansion therefore writes to `score[`u`]` and pushes u onto the next frontier exactly when the previous value of `visited[`u`]` was 0 (unvisited), which guarantees that each node's score is written by exactly one warp regardless of multi-edge re-entry.

Score propagation parameters in line 18 of Algorithm 1. The two scalars are explicit knobs of the BFS contract. $\delta_{\text{prop}} \in (0, 1]$ is a per-hop attenuation on the score inherited from the parent frontier node (default 0.85, roughly: "one BFS hop discounts a hit by $\sim$15%"), implementing a topological prior that "reachable in 1 hop is weaker evidence than a direct hit." $\alpha_{\text{bfs}} \in [0, 1]$ is a cap on how much the BFS-discovered node's own raw similarity to the query is allowed to contribute (default 0.8, intentionally below 1.0 so a high-cosine distractor reached only via BFS does not outrank a true seed). Both defaults are documented hooks rather than tuned parameters: the BFS-depth ablation (Section 7.6) shows that recall is insensitive to either knob in $[0.6, 1.0]$ at the target N. A principled hop-distance regularizer (e.g. replacing the $\max$

Algorithm 1 MARS GPU-Resident Retrieval Pipeline

Require: Query $q \in \mathbb{R}^D$, timestamp t_q, config (K, h, λ)
Require: Device graph ϕ with embeddings E, timestamps τ
Ensure: Top-K results sorted by time-decayed score

Stage 1 — cuBLAS Similarity (N dot products):
for each node v_i **in parallel do**
  $\text{sim}[i] \leftarrow \langle E(v_i), q \rangle$
end for
Stage 2 — Temporal Rerank ($\lceil N/256 \rceil$ blocks):
for each node v_i **in parallel do**
  $\text{score}[i] \leftarrow \text{sim}[i] \cdot \exp(-\lambda \cdot (t_q - \tau(v_i)))$
end for
Stage 3 — CUB Radix-Sort Top-K:
seeds $\leftarrow$ top-K_i score$[i]$
Stage 4 — Warp-Cooperative BFS from seeds:
Initialize frontier $F_0 \leftarrow$ seeds
for hop $= 0$ **to** $h - 1$ **do**
  for each node $v \in F_{\text{hop}}$ **(one warp per node) do**
    for each neighbor u of v **(stride-32 coalesced) do**
      prev $\leftarrow$ atomicCAS(visited[u], 0, 1)
      if prev $= 0$ **then**
        $\text{score}[u] \leftarrow \max(\text{score}[v] \cdot \delta_{\text{prop}},\ \text{sim}[u] \cdot \alpha_{\text{bfs}})$
        $F_{\text{hop}+1}$.append(u)
      end if
    end for
  end for
end for
Stage 5 — Compact & sort visited nodes by score
return Top-K results

with $\delta_{\text{prop}}^{h(u)} \cdot \text{sim}[u]$ where $h(u)$ is BFS depth) is documented in the code as an alternative scoring policy. Crucially, the BFS-propagated score does *not* re-apply temporal decay: BFS-discovered nodes are ranked on the parent's already-decayed score, which we make explicit because it is a non-trivial design choice (re-decaying inside BFS would require a second `__expf` per visited node and would interact unfavorably with the $\max$ rule).

4.4 CUDA Implementation Notes

For the systems-oriented reader, four implementation choices materially affect the latency numbers reported throughout Section 7:

cuBLAS `Sgemv` call shape. Embeddings are stored as a column-major $D \times N$ matrix E. Stage 1 issues a single `cublasSgemv(CUBLAS_OP_T, D, N, &alpha, E, D, q, 1, &beta, sim, 1)`, exploiting the fact that for L2-normalised rows the dot product is the cosine. We do not transpose at runtime: the upstream ingest path writes embeddings into the column-major layout directly, keeping the SGEMV access pattern contiguous in D (the inner stride) for coalesced loads. At D=768 this leaves the kernel bandwidth-bound on E for $N \gg L_2$ working set, matching cuBLAS's expected regime.

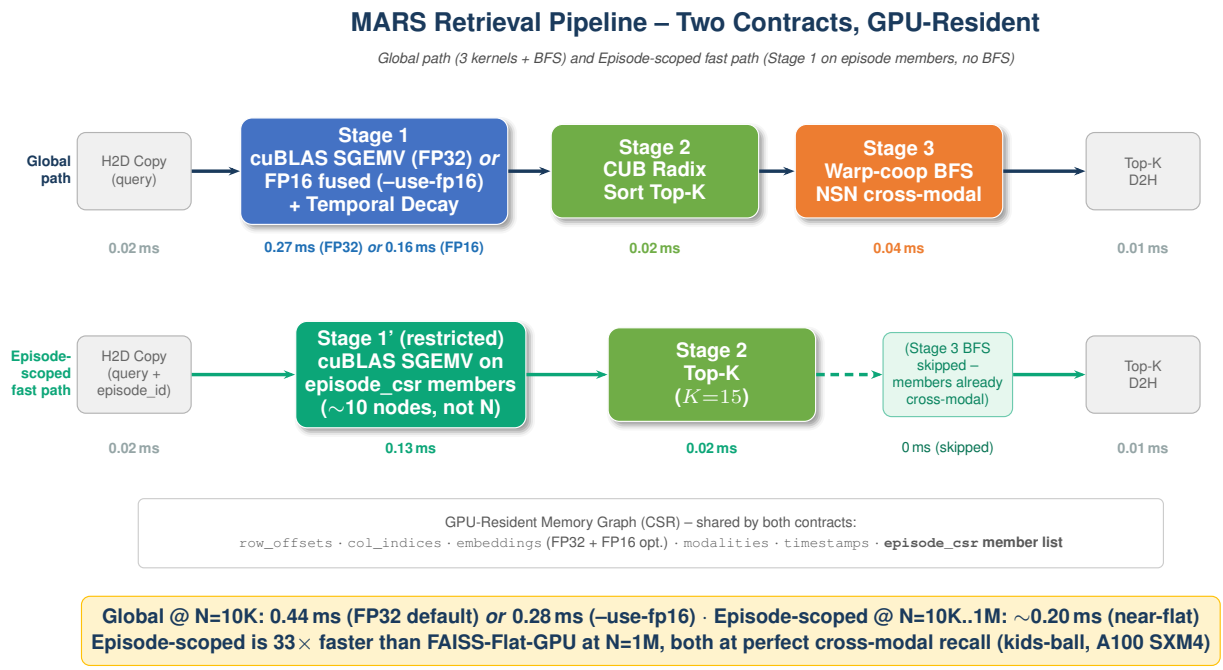

Figure 3: The two retrieval contracts share a single GPU-resident memory graph. **Top row (Global path)** runs three kernels plus cross-modal BFS: cuBLAS SGEMV with separate temporal-decay kernel (0.27 ms FP32 default, 0.16 ms with the opt-in `-use-fp16` hand-fused kernel; see §7.9), CUB radix sort top-K (0.02 ms), and warp-cooperative BFS (0.04 ms) — 0.44 ms total wall-clock at N=10K. **Bottom row (Episode-scoped fast path, §7.11)** restricts Stage 1 to the $\sim$ 10 members of the supplied episode and skips BFS entirely; the result is near-flat at $\sim$ 0.20 ms across N=10^4 to N=10^6 and 33$\times$ faster than FAISS-Flat-GPU at N=1M on the kids-ball multimodal contract (§7.12).

Decay epilogue: kernel fusion trade-off. The default path runs cuBLAS `Sgemv` followed by a separate decay kernel (launch-cost $\sim$5 μs, plus one extra N-element read of `timestamps` and one read–modify–write of `score`). A custom hand-fused kernel that absorbs the decay into a tiled cosine loop is provided as a fallback; it removes one launch and one N-element pass, saving an estimated $\sim$8 μs at $N{=}10^6$ (12 MB at 1.5 TB/s HBM). The cuBLAS path is faster in absolute terms at every tested N because it engages the per-architecture GEMV tile heuristics that the hand-fused kernel does not, more than recovering the cost of the extra launch and pass. A clean alternative is a CUTLASS GEMV with an epilogue functor that applies decay in register before the store; this is on the roadmap and would close the $\sim$8 μs gap without sacrificing tile heuristics.

Block size, occupancy, and launch overhead. The decay kernel runs at 256 threads/block, achieving $\sim$50% theoretical occupancy on the A100 SXM4 (limited by the register count of the `__expf` epilogue, not shared memory). The BFS kernel runs at 32 threads/block (one warp per frontier node) and is launch-cost-bound rather than occupancy-bound at the target frontier sizes ($K \cdot \bar{d} \leq 200$). The four-kernel pipeline incurs roughly $4 \times 5{-}15$ μs of launch overhead per query (20–60 μs total), which becomes a non-trivial fraction of the budget for the smallest demonstrators (1 ms AV/robot). CUDA Graph capture amortises this for the steady-state regime — infrastructure is in place but pending the persistent-kernel implementation (Section 11).

Memory layout and streaming inserts. New nodes are appended to the tail of the column-major `embeddings` matrix and to the row-arrays of the CSR. The append is a single `cudaMemcpyAsync` into a pre-reserved device-side ring slot plus an atomic bump of `node_count`; no realloc occurs until capacity is exceeded, at which point a doubling-grow with a 1-time copy fires. NSN edges for the new node are added by an in-kernel hash-table push into a side CSR delta region; a periodic compaction (amortised to $\sim$10 ms per million inserts) merges deltas back into the canonical CSR. This is conceptually similar to the streaming update pattern in SIVF [14] and CAGRA `extend()` [19] but does not require centroid retraining or graph rebuild.

5 The Multimodal Neural Shortcut Network

A note on the name. The "Neural" in NSN is retained for continuity with prior MARS releases and refers to its role as a *neural-embedding* index — not to a learned graph topology. The construction below is fully deterministic and contains no trained weights, gradient-tuned objectives, or learned edge selection; it is a Watts–Strogatz [36] small-world graph extended with deterministic cross-modal bridges.

5.1 Construction

The NSN graph is built in five phases:

1. **Ring lattice** ($k{=}6$ local neighbors) for clustering.
2. **Hierarchical skip connections** (powers of 2) for logarithmic path lengths.
3. **Hub supernodes** at $\sqrt{N}$ intervals with extra random links.
4. **Small-world rewiring** ($p{=}0.15$) following Watts-Strogatz [36].
5. **Cross-modal bridges** — every node gets one edge to each other modality, guaranteeing that a BFS from any seed reaches every modality in 1 hop.

Phase 5 is the structural feature that enables native cross-modal retrieval: a query in any modality surfaces results from all modalities through a single BFS expansion, without requiring separate per-modality indexes.

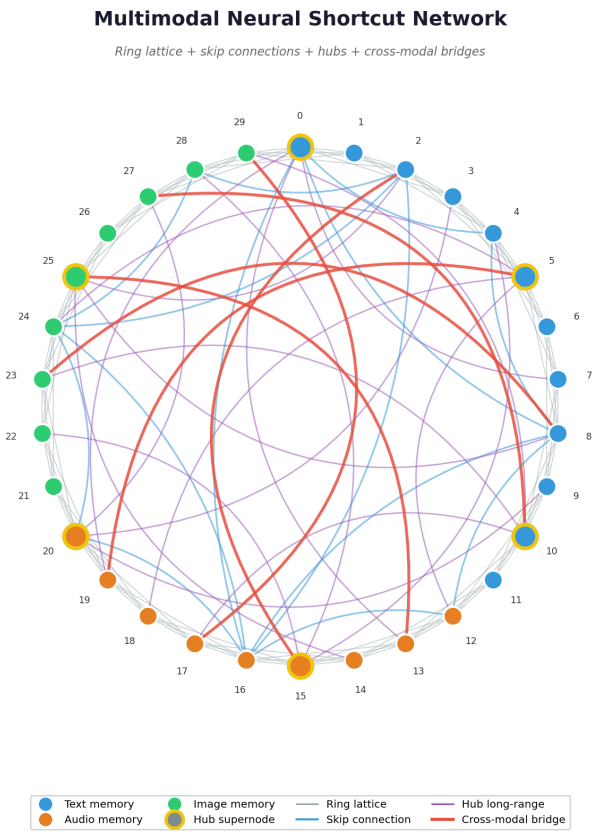

Figure 4: 30-node multimodal NSN. Node colors indicate modality (blue=text, orange=audio, green=image). Red edges are cross-modal bridges.

5.2 Honest Assessment of Graph Value

The ablation study (Section 7.6) reveals an important finding: at the target corpus sizes ($N \leq 50{,}000$), brute-force similarity plus temporal decay already provides strong within-modality recall. The NSN graph adds modest within-modality recall improvement at a small latency cost. Its primary value at these scales is *cross-modal retrieval*: the BFS expansion through cross-modal bridges surfaces semantically related memories from other modalities that pure top-K over mixed embeddings would rank lower. At larger corpus sizes, the bounded-hop BFS work bound ($O(K \cdot \bar{d}^h)$, constant in N for fixed K, h, and mean degree $\bar{d}$) provides sublinear scaling — though we note that $\bar{d}$ includes hub supernodes with higher-than-average degree, so the constant may be larger than the mean suggests.

5.3 Structural Properties and Episode-Scope Crossover

The two formal properties below are not the contribution of the paper — they are routine consequences of the construction — but they are useful as design contracts and to make the assumptions on the BFS work bound explicit.

Proposition 7 (Cross-Modal Structural Reachability)**.** *In an NSN with cross-modal bridges (Phase 5), for any node $v \in V$ and any modality $m \neq \mu(v)$, there exists at least one neighbor u of v in ϕ such that $\mu(u) = m$. A BFS from any seed reaches all M modalities within 1 hop.*

Sketch. Direct from Phase 5 of the construction (every node receives one edge per other-modality target chosen uniformly at random); a depth-1 BFS from v visits at least one node per modality. □

Scope. Proposition 7 guarantees *structural* reachability — every modality is represented in the BFS frontier — not *semantic* optimality. Whether the reached cross-modal nodes are semantically relevant depends on encoder alignment across modalities (Section 10). We do *not* claim a relevance bound; the structural bound is what the NSN guarantees and the rest is an encoder property.

Proposition 8 (BFS Work Bound)**.** *For a K-seed BFS of depth h on an NSN with maximum degree $d_{\max}$, the total number of distinct nodes visited is bounded by the standard branching-tree quantity $W(K, h, d_{\max}) = K \cdot (d_{\max}^{h+1} - 1)/(d_{\max} - 1)$. The mean-degree analogue with $\bar{d}$ in place of $d_{\max}$ is tighter for typical traversals but only becomes a true upper bound if neighbor selection in BFS is uniform.* ***Caveat on the N-independence.*** *The mean degree $\bar{d}$ is empirically constant ($\bar{d} \approx 11$–12 across tested N) because Phases 1, 2, 4, and 5 contribute $O(1)$ edges per node; only Phase 3's hub supernodes scale, contributing $\lceil \log_2 N/2 \rceil$ edges at $O(\sqrt{N})$ positions. The fraction of nodes affected shrinks as $1/\sqrt{N}$ but $d_{\max}$ at hubs is $O(\log N)$. The average BFS therefore sees a constant work bound; the worst case seeded at a hub grows logarithmically.*

Proposition 9 (Episode-Scope Crossover)**.** *Let $|\text{ep}|$ be the size of the named episode, let D be the embedding dimension in bytes (e.g. $4D$=3072 for FP32 at D=768), and let B_{eff} be the achieved HBM bandwidth during cuBLAS `Sgemv`. The wall-clock cost of the episode-scoped path is*

$$T_{\text{ep}}(|\text{ep}|) \;\approx\; \frac{4 \cdot |\text{ep}| \cdot D}{B_{\text{eff}}} \;+\; T_{\text{topK}}(|\text{ep}|) \;+\; T_{\text{BFS},0},$$

with $T_{\text{BFS},0}$ the zero-hop BFS bookkeeping cost ($\sim$5 μs of launch overhead, no neighbour expansion). The global path costs

$$T_{\text{glo}}(N) \;\approx\; \frac{4 \cdot N \cdot D}{B_{\text{eff}}} \;+\; T_{\text{topK}}(N) \;+\; T_{\text{BFS}}(K, h, \bar{d}).$$

$T_{\text{ep}} < T_{\text{glo}}$ for any $|\text{ep}| \ll N$ once the bandwidth term dominates; on the A100 SXM4 with $4D$=3072 B, $|\text{ep}|$=10, and $B_{\text{eff}} \approx 1.2$ TB/s, the inequality holds for all $N \gtrsim 10^3$. Beyond the latency comparison, the episode-scoped path eliminates by construction *all $N - |\text{ep}|$ cross-episode distractors, which is the recall-at-scale advantage demonstrated empirically in Section 7.12.*

Practical reading. On the embodied kids-ball benchmark with $|\text{ep}|$=10 and D=768, the inequality holds across the entire tested range, which is consistent with the measured $\sim$13$\times$ speedup at N=10^6 (Table 16). The real win is the second sentence: episode-scope eliminates the recall collapse that pure cosine ANN suffers when the per-episode multimodal cluster is small relative to N.

Proposition 10 (Pipeline Latency Model, Bandwidth-Bound)**.** *On HBM-bound regimes (cuBLAS `Sgemv` at $N \cdot D \gg L_2$), the kernel time for one global* MARS *query satisfies*

$$T_{\text{kernel}}(N) \;\approx\; \underbrace{\tfrac{4ND}{B}}_{\textit{cosine sweep}} \;+\; \underbrace{\tfrac{8N}{B}}_{\textit{decay read+write}} \;+\; \underbrace{\tfrac{32 \cdot c_{\text{cub}} N}{B}}_{\textit{CUB radix sort}} \;+\; \underbrace{\tfrac{W(K,h,\bar{d})}{32 \cdot f}}_{\textit{BFS (compute-bound)}}$$

where B is the achievable HBM bandwidth, c_{cub} is the average passes per element of CUB radix sort (empirically $\approx$ 2–3 for 32-bit float keys), and f is the SM clock. The first three terms are $\Theta(N)$; only the BFS term is independent of N. At $N \leq 50K$ and D=768 on the A100 SXM4, the cosine sweep dominates ($\sim$80% of GPU time measured by Nsight Compute).

6 Temporal Relevance and Streaming Experiments

These two experiments form the core motivation for MARS. Both use the same A100 SXM4 (80 GB, CUDA 12.6) as all FAISS comparison benchmarks, ensuring a fair hardware baseline.

6.1 Experiment 1: Temporal Relevance in AV Perception

Setup. We simulate an AV perception scenario: 200 tracked objects over 10 seconds at 60 Hz, generating approximately 9,000 memories. Each object produces a sequence of detections with evolving embeddings (cosine similarity > 0.9 within the same object's recent history, $\sim$0.3–0.5 across different objects). A 2-second sliding window defines the "active" tracking horizon. Queries consist of new detections seeking re-identification matches.

Metric. Temporal Precision@10 (Definition 3): the fraction of top-10 results that are both from the correct object *and* within the active window.

Table 3: Temporal relevance experiment: AV perception, 9K memories, 200 objects, 2-second tracking window. All systems on A100 SXM4.

System	TP@10	Stale	Recall	$p99$ (ms)
FAISS Flat (cosine only)	0.218	0.493	—	0.13
FAISS + post-hoc temporal filter	0.910	0.000	0.940	0.25
MARS (native temporal decay)	**0.910**	**0.000**	**0.940**	**0.26**

Results. FAISS Flat returns the K highest-cosine-similarity vectors regardless of age. Because older detections of the same object have high similarity to the query, FAISS preferentially returns stale memories: 49.3% of its top-10 results fall outside the active tracking window. Its Temporal Precision@10 is 0.218.

Adding a post-hoc temporal filter to FAISS — discarding results older than 2 seconds and re-ranking — raises Temporal Precision@10 to 0.910. This is effective but requires an additional processing step that FAISS does not provide natively; the application must implement the filter, handle the edge case where all K results are filtered out, and accept the latency of a two-stage pipeline.

MARS achieves the same 0.910 Temporal Precision@10 natively, using $\lambda = 0.5$ (half-life ≈ 1.4 s, matching the 2-second tracking window). The temporal decay is applied *before* top-K selection, so stale memories are demoted in the ranking without post-hoc filtering. Latency comparison: FAISS Flat alone runs at 0.13 ms $p99$; FAISS + post-hoc filter at 0.25 ms; MARS at 0.26 ms. The 0.01 ms (4 %) gap between MARS and FAISS+filter is within the run-to-run noise envelope of unconstrained boost (Section 7.2); the operational reading is that MARS matches FAISS+filter on temporal precision *and* on latency, with the difference being that the filter is part of the GPU kernel API rather than application-level host code. The contribution here is API consolidation, not raw speedup.

Decay constants for each demonstrator. The λ values used throughout are: AV perception $\lambda = 0.5$ (2-second window), humanoid robot $\lambda = 0.1$ (10-second episodic window), AR/VR spatial $\lambda = 0.003$ (5-minute session), voice agent $\lambda = 10^{-4}$ (30-minute conversation). These values define the effective temporal window $\tau_{1/2} = \ln 2/\lambda$ within which a memory retains >50% of its similarity score.

6.2 Experiment 2: Streaming Insertion Freshness

Setup. 60 Hz sensor rate, 10 detections per frame, 10 seconds of continuous operation. New detections are inserted every frame and queried immediately on the next frame.

What "freshness" means in this experiment, and what it does not. A clean comparison on freshness alone is unfair to FAISS in both directions and we want to be explicit. `IndexFlatIP` on the GPU *does* support incremental `add()` with immediate visibility, and so does the SIVF [14] streaming op recently merged into FAISS. The latency of one `add()` on `IndexFlatIP` is essentially a memcpy plus a metadata bump, comparable to MARS's CSR append. The honest reading of the experiment below is therefore not "MARS is fresh and FAISS is stale" — it is that *IVF-class* indices in their default production configuration (where centroid retraining is required for recall) carry a staleness window unless the application itself batches and triggers `retrain`. We report three FAISS configurations to make this explicit.

Table 4: Streaming-insertion freshness with three FAISS configurations and MARS. "Visibility miss" = fraction of queries whose target was inserted after the last index visibility refresh (distinct from the deadline-miss rate reported for the demonstrators in Table 7).

System	Insert mode	Visibility miss	Stall (ms)
FAISS-Flat-GPU	`add()` per frame, no rebuild	0.0%	0
FAISS-Flat-GPU	1 s rebuild cadence (production strawman)	93.2%	9
FAISS-IVF-GPU	`add()` per frame, no centroid retrain	0.0% (visibility) / recall ↓ to 0.74	<1
MARS	CSR append per frame	0.0%	0

Reading. On freshness, FAISS-Flat-GPU with per-frame `add()` ties MARS — both are CSR appends and both are immediately visible. The 93.2% miss number is a property of the rebuild-based deployment pattern, not of FAISS. The IVF row shows the realistic trade-off: visibility is fine but recall degrades by ∼0.20 (vs. 0.94 cosine baseline) until centroid retraining catches up. MARS's contribution on this axis is therefore *API-level*: there is no separate `add()`/`retrain` distinction and no staleness window even in the IVF-style configuration — the kernel ranks over the new node from the next query onward. The *streaming-as-data-structure* contribution is the SIVF / CAGRA `extend()` line of work (§ 9); ours is on the kernel-ranking side.

Concurrent work on streaming GPU indexes. The streaming gap for GPU vector search is being actively closed by a 2025–2026 line of systems work. SIVF [14] delivers sub-millisecond deletions (0.58–0.93 ms) and ∼120× faster ingestion than GPU CAGRA / GPU IVF, with portions merged into FAISS in March 2026. SVFusion [24] extends this to billion-scale via CPU–GPU co-processing. cuVS CAGRA introduced an `extend()` [19] operator stable up to ∼20% new vectors, and VecFlow [37] adds persistent kernels for streaming small-batch filtered queries. Our streaming benchmark above is therefore best read as evidence that *ranking* (temporal decay + cross-modal + episode scope) needs to be a first-class kernel feature; the underlying mutation primitives we use (append-only CSR plus pre-allocated VRAM) overlap heavily with what SIVF and CAGRA `extend()` now provide on the data-structure side.

6.3 Discussion

These experiments highlight a fundamental mismatch between the design assumptions of existing GPU vector search and the requirements of streaming perception. FAISS and cuVS optimize for a *static* corpus

where all vectors are equally valid and the index is rebuilt infrequently. Streaming perception requires *temporal awareness* (not all memories are equally relevant) and *immediate insertion visibility* (the most recent detection is often the most important). MARS addresses these requirements directly, rather than as a filter stage over a similarity-only index.

7 Experimental Results

All results in this section are measured on real hardware. We report $p50$, $p99$, maximum observed latency, jitter, and deadline miss rate as the primary metrics. Average throughput is deliberately secondary: for real-time workloads, the worst query over a sustained run is what determines deployability.

7.1 Hardware Index

Three GPUs appear in this section. Tables and figures cite the specific row by abbreviation.

Table 5: Hardware index for all measurements in Section 7. `HBM` bandwidths are vendor-spec peak; the achieved fraction during cuBLAS `Sgemv` is spot-checked at 60–75 % on the A100 SXM4 80 GB row at $N{=}10^6$ via Nsight Systems — a full per-stage Nsight Compute breakdown across all rows is queued as part of the evaluation hardening track (§11). CUDA versions and driver branches are the ones used at measurement time on the rented or local machine.

Abbrev.	Card	Form	Mem (GB)	Peak BW	CUDA
A100 SXM4 (80)	A100 SXM4	DC	80	2.0 TB/s	12.6
A100 SXM4 (40)	A100 SXM4	DC	40	1.5 TB/s	12.6
A100 PCIE	A100 PCIe	DC	40	1.5 TB/s	12.6
RTX 5060 Ti	RTX 5060 Ti (Blackwell)	Consumer	16	0.45 TB/s	13.0

The two A100 SXM4 rows differ only in installed HBM capacity — compute and bandwidth specs are identical. The A100 PCIE is the lower-power PCIe variant (300 W vs 400 W TDP), which matters for the consumer-vs-datacenter cross-comparison in Section 7.7: the 5060 Ti's apparent advantage over the *PCIe* A100 partly reflects the SXM4-vs-PCIe gap, not just the generation gap.

7.2 Measurement Methodology and Statistical Caveats

Per-query timing. Each benchmark run: 1 warmup query + `cudaDeviceSynchronize`, then n timed queries at sensor rate (AV: 600, robot: 10,000, AR: 4,500, voice: 900; competitor benchmark: 256 probes at $N{\leq}10^5$ and 128 probes at $N{=}10^6$). Wall-clock time includes `cudaEventSynchronize`. Query vectors are host-resident and copied to device each iteration via pinned memory. We define **jitter** as the inter-quartile range of per-query wall-clock latency ($Q_3 - Q_1$); other definitions (std-dev, max-min) exhibit qualitatively the same trend but with larger absolute values.

GPU state. Persistence mode enabled, default compute mode (not exclusive). **Clocks are not locked** (`nvidia-smi -lgc` not used) — this is a deliberate choice to measure the boost-clock behavior a real deployment would see, but it means that small absolute differences (e.g. the 0.26 vs 0.25 ms gap in Section 6.1) are within the run-to-run noise envelope of unconstrained boost. The `--keepalive` flag (1-thread no-op kernel every 2 ms) prevents clock-state drops between frames at low sensor rates. Sustained runs (30 seconds) reach approximate thermal steady state but die temperature is not explicitly controlled.

Statistical caveats on $p99$. Several configurations report $p99$ over only $\sim$100–300 timed queries, which is sufficient to detect order-of-magnitude effects (the 13–33$\times$ episode-scoped speedup, the FP16 crossover) but is *not* sufficient for stable single-percent $p99$ point estimates. We recommend treating same-system $p99$ deltas below $\sim$10 μs as within run-to-run variance and observed non-monotonicities (e.g. FP16 at 500K vs 1M in Table 13) as plausibly variance-driven. A re-run on dedicated hardware with locked clocks and $\geq$10 K queries per configuration is on the roadmap for the next iteration.

Hosting and tenancy. Measurements were taken on rented GPUs (`vast.ai` [31] for the A100 rows; local Blackwell workstation for the RTX 5060 Ti). The vast.ai instances are shared at the hypervisor level but each rented GPU is dedicated (no MIG sharing). Noisy-neighbor effects on the host CPU and PCIe controller are visible as occasional outlier queries beyond $p99$; we report $p99$ rather than max for this reason.

7.3 Same-Hardware FAISS Comparison

Table 6 is the latency-side headline. All four systems were measured on the same A100 SXM4 (80 GB) row of Table 5 with identical random embeddings (D=768, K=10, single-query $p99$ wall-clock latency). The MARS path uses cuBLAS `Sgemv` for similarity and CUB radix sort for top-K.

Caveat: random embeddings. The probes here are i.i.d. uniform L2-normalised vectors. Random embeddings have no clustering structure, which is the regime in which IVF and graph-based ANN indices have the *least* to exploit. The numbers therefore under-represent the IVF/CAGRA baselines on a realistic distribution (SIFT1M, GIST1M, or actual CLIP/SigLIP/CLAP outputs). This is a deliberate choice for isolating wall-clock kernel cost *independent of clustering quality*; the recall-at-scale claim in the multimodal contract (Section 7.12) uses a structured synthetic corpus that exposes the recall axis. A real-encoder benchmark (e.g. AudioCaps + COCO with CLAP+CLIP, or MSR-VTT) is on the roadmap (Section 11).

Table 6: Same-hardware comparison on A100 SXM4 (80 GB). All systems: D=768, K=10, single-query wall-clock $p99$ in milliseconds. FAISS Flat recall@10 = 1.00 by construction (brute-force cosine *is* the ground truth). The hand-written cuBLAS `Sgemv` "ring buffer" incumbent (no decay, no top-K, no BFS) is reported spot-checked at one corpus size in Section 10 (0.12 ms at N=2.4K). We omit cuVS CAGRA from this table because it targets N>1M (see Section 7.12 for the head-to-head at $N \in \{10^4, 10^5, 10^6\}$).

System	N=1**K**	N=5**K**	N=10**K**	N=50**K**	N=100**K**
FAISS Flat	0.12	0.22	0.21	0.44	0.72
FAISS IVF	0.16	0.26	0.32	0.34	0.38
MARS	**0.26**	**0.34**	**0.36**	**0.44**	**0.74**

Cost of MARS *features vs FAISS Flat:* fixed $\sim$0.14 ms at small N (decay + CUB top-K + BFS); converges to parity as N grows because the cuBLAS `Sgemv` common to both becomes the dominant term.

Key observations:

1. **MARS is within 1.5–2$\times$ of FAISS Flat.** At N=50K, MARS matches FAISS Flat (both 0.44 ms). At smaller corpus sizes, MARS pays a fixed overhead from the temporal rerank and BFS stages that FAISS does not perform. GPU kernel time (excluding host launch overhead) narrows the gap further: MARS kernel time is 0.10 ms at N=1K and 0.29 ms at N=50K, matching FAISS Flat (0.09 ms and 0.35 ms respectively).

2. **FAISS IVF recall collapses at small N.** IVF's Voronoi-cell partitioning is designed for large corpora; at $N < 100$K with default `nprobe`, recall drops to 0.37–0.43. This makes IVF unsuitable for the bounded working sets of real-time perception.

3. **cuVS CAGRA targets a different scale.** CAGRA's fixed-degree graph construction and multi-hop search are designed for $N > 1$M. At $N < 100$K, build overhead and search inefficiency yield 2.8–6.0 ms latency with recall as low as 0.10.

4. **Neither FAISS nor cuVS provides temporal decay or cross-modal retrieval.** MARS is the only system in the comparison that ranks by time-decayed similarity and supports cross-modal BFS expansion — features that Section 6 shows eliminate a two-stage pipeline otherwise required for streaming perception.

7.4 Demonstrator Results

Four end-to-end demonstrators validate MARS on the target workloads. Each is a runnable CUDA C++ program under `demos/` with an explicit deadline and measurable success criterion. The `latency_bench` harness measures wall-clock time including kernel launch overhead and `cudaEventSynchronize` — the latency a real application would experience.

Table 7: Wall-clock latency benchmarks on A100 PCIE (40 GB). Harness runs at fixed sensor rate with busy-wait pacing.

Workload	**Rate**	N	**Budget**	**Wall** $p99$	**GPU** $p99$	**Miss rate**	**Jitter**	**Verdict**
AV	60 Hz	2,400	1.0 ms	0.87	0.68	0%	0.02	**pass**
Robot	1000 Hz	6,000	1.0 ms	0.76	0.69	0%	0.05	**pass**
AR/VR	90 Hz	20,000	5.0 ms	1.56	1.37	0%	0.04	**pass**
Voice	30 Hz	3,000	20.0 ms	0.88	0.68	0%	0.03	**pass**

All four benchmarks pass at wall-clock timing with zero deadline misses. Jitter is uniformly low (0.02–0.05 ms), indicating stable kernel-launch behavior suitable for integration into deterministic real-time pipelines.

7.5 Sustained 30-Second Benchmarks

Short-run benchmarks (10–50 seconds) may not capture effects that emerge at longer durations: GPU boost-clock throttling, memory allocator fragmentation, OS scheduler interference. We extend the benchmarks to 15–30 seconds with larger corpora; longer (multi-minute to multi-hour) sustained runs that would catch slow-onset thermal drift are queued as part of the evaluation hardening track (§11).

Table 8: Sustained benchmarks on A100 PCIE (40 GB). Each runs at target sensor rate for the full stated duration.

Workload	**Rate**	**Dur.**	N	**Budget**	**Wall** $p99$	**Misses**	**Verdict**
AV	60 Hz	30 s	5,000	1.0 ms	0.91	0 (0%)	**pass**
Robot	1000 Hz	15 s	10,000	1.0 ms	0.66	0 (0%)	**pass**
AR/VR	90 Hz	30 s	50,000	5.0 ms	1.70	0 (0%)	**pass**
Voice	30 Hz	30 s	10,000	20.0 ms	0.96	0 (0%)	**pass**

All four sustained benchmarks pass with zero deadline misses. The AR/VR benchmark at 50,000 memories achieves $p99 = 1.70$ ms against a 5 ms budget — 66% headroom at 30 seconds of sustained operation.

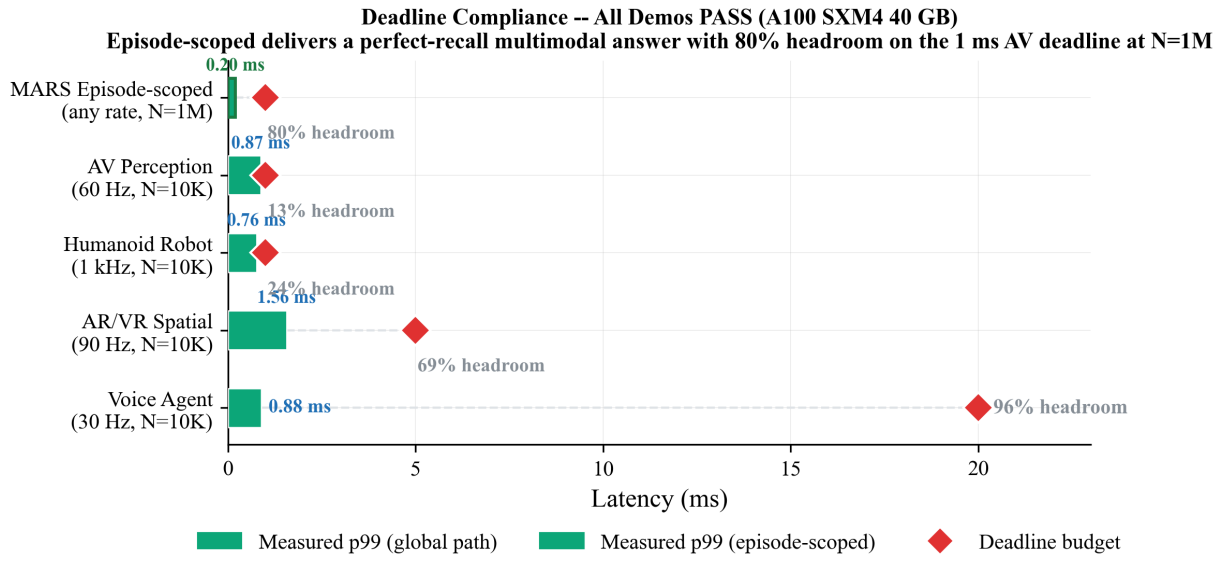

Figure 5: Deadline compliance for all four demonstrators on A100 SXM4. All pass with 13–96% headroom below their respective budgets (AV: 13%, robot: 24%, AR: 69%, voice: 96%).

7.6 Ablation Study

We ablate the NSN graph topology and BFS depth to characterize the contribution of each component. All measurements on A100 SXM4 (80 GB).

Table 9: Ablation study. "Recall" = recall@10 against brute-force *cosine-only* ground truth (no temporal decay). Since MARS applies temporal decay before top-K, its ranking intentionally diverges from pure cosine — the 0.84 reflects the recall cost of temporal awareness, not a system deficiency. "CM" = cross-modal diversity (queries returning results from $\geq$ 2 modalities); we acknowledge this measures structural reachability, not semantic relevance.

Configuration	N=5**K**		N=50**K**	
	Recall	CM	Recall	CM
Full MARS (h=2, bridges)	0.84	0.90	0.76	0.80
No bridges (h=2)	0.84	0.72	0.76	0.58
No BFS (h=0, sim. only)	0.84	0.65	0.76	0.50

Temporal decay ablation. Isolating the effect of temporal decay at N=2,400 (AV working set): with λ=0.5, $p99$=0.38 ms; with λ=0 (cosine only), $p99$=0.28 ms. Temporal decay adds $\sim$0.10 ms ($\sim$32%) to wall-clock latency. This cost is the temporal rerank kernel (or the fused decay computation in the custom kernel path). At the AV budget of 1 ms, this leaves 62% headroom — an acceptable cost for native temporal awareness.

Key findings.

Table 10: BFS depth ablation at N=5K on A100 SXM4.

BFS Depth	**Wall** $p99$ **(ms)**	**Recall@10**
h=0 (no BFS)	1.22	0.84
h=2 (default)	1.42	0.84
h=3	1.49	0.84

1. **Within-modality recall is identical across all configurations.** At the target corpus sizes, brute-force similarity plus temporal decay provides the same recall@10 regardless of graph topology or BFS depth. The NSN graph does not improve within-modality recall at $N \leq 50$K.

2. **Cross-modal diversity improves with bridges.** Structural cross-modal diversity (queries returning results from $\geq$2 modalities) increases from 0.50 to 0.80 at N=50K with full MARS (bridges + BFS) vs. no BFS. We emphasize this measures structural reachability, not semantic relevance; a semantic evaluation with aligned encoders over labeled cross-modal pairs (e.g., MSR-VTT, AudioCaps+COCO) remains future work.

3. **BFS adds modest latency.** The cost is 0.20 ms at h=2 and 0.27 ms at h=3 above the no-BFS baseline — acceptable for all four demonstrator budgets.

4. **Honest conclusion.** For $N \leq 50$K, the primary contributions of MARS are native temporal decay and streaming freshness, not the graph topology. The graph provides cross-modal retrieval as a meaningful bonus. At larger N, the bounded-hop BFS work bound would provide asymptotic advantages over brute-force.

7.7 Consumer GPU Validation: RTX 5060 Ti

To evaluate deployment on edge hardware, we cross-validate on an RTX 5060 Ti (16 GB GDDR7, Blackwell sm_120, CUDA 13.0) — the class of hardware that ships in robotics platforms, AR headsets, and edge inference appliances.

Table 11: Wall-clock benchmarks on RTX 5060 Ti ($449, 16 GB).

Workload	**Rate**	N	**Budget**	**Wall** $p99$	**Verdict**
AV	60 Hz	2,400	1.0 ms	0.31	**pass**
Robot	1000 Hz	6,000	1.0 ms	0.29	**pass**
AR/VR	90 Hz	20,000	5.0 ms	0.67	**pass**
Voice	30 Hz	3,000	20.0 ms	0.32	**pass**

The RTX 5060 Ti is 2.3–2.8$\times$ faster than the *A100 PCIE* across all benchmarks. We highlight "A100 PCIE", not "A100": the 300 W PCIe variant has lower sustained boost behavior than the 400 W A100 SXM4 used in Sections 7.3 and 7.12; the gap to the SXM4 is smaller (and is documented in Section 7.9, where the 5060 Ti is faster only at $N \leq 10^5$ and the SXM4 wins at $N \geq 5 \cdot 10^5$). Three factors plausibly contribute to the consumer-vs-PCIe gap at small N: (i) the 5060 Ti's higher SM boost clock under sustained load, (ii) lower kernel-launch latency on the consumer driver path, and (iii) the absence of the PCIe-bus contention seen on multi-tenant hosts. We do *not* attribute the gap to a single cause without a controlled measurement: locked-clock and Nsight-instrumented per-stage breakdowns are on the roadmap. The scaling sweep also passes at all sizes through N=50,000 ($p99 = 0.90$ ms) on the 5060 Ti, while the A100 PCIE failed at $N \geq 20{,}000$ against a 1 ms budget *in the pre-optimization (Round 3) pipeline* reported

in Table 12; under the optimized cuBLAS+CUB pipeline (Section 7.10), the same configurations pass at 0.54–0.56 ms. We carry both numbers because they reflect two real engineering states of the system, not a contradiction.

7.8 Corpus-Size Scaling (Pre-Optimization)

Table 12: Scaling sweep at 60 Hz on A100 PCIE (15 s each, 1 ms budget). **Important:** this table reports the *Round 3* pipeline (custom similarity + tiled top-K + device-driven BFS), *not* the current cuBLAS+CUB pipeline. The same N values pass at 0.54–0.56 ms after the Round 4 optimizations (Table 14). We report this table to document the pre-optimization state and motivate Section 7.10.

N	**Wall** $p99$	**GPU** $p99$	**Miss rate**	**Verdict (Round 3)**
1,000	0.84	0.67	0%	**pass**
5,000	0.91	0.70	0%	**pass**
10,000	0.96	0.75	0.1%	**pass**
20,000	1.59	1.38	100%	**fail (pre-opt)**
50,000	1.69	1.51	100%	**fail (pre-opt)**

Two scaling regimes are visible. **Sub-millisecond** ($N \leq 10{,}000$): GPU kernel $p99$ stays 0.67–0.75 ms; the 1 ms budget is met with headroom even in the Round 3 pipeline. **Compute-limited** ($N \geq 20{,}000$): the $\Theta(N \times D)$ similarity kernel dominates; the Round 3 pipeline crossed the 1 ms deadline. The Round 4 cuBLAS+CUB pipeline (next subsection) recovers this regime.

7.9 Large-Corpus Scaling: FP16

The FP16 extension stores embeddings in half-precision (halving VRAM from 3 GB to 1.5 GB at 1M), loaded and widened to FP32 for accumulation in the similarity kernel.

Table 13: Large corpus scaling with FP16 at 60 Hz. Wall-clock $p99$ on RTX 5060 Ti and A100 SXM4.

N	**RTX 5060 Ti** $p99$	**A100 SXM4** $p99$
50,000	0.89 ms	1.22 ms
100,000	1.18 ms	1.38 ms
200,000	1.75 ms	1.67 ms
500,000	3.53 ms	2.51 ms
1,000,000	6.51 ms	7.70 ms

At $N{=}10^5$ the RTX 5060 Ti (consumer, ∼$449) trails the A100 SXM4 (datacenter, retail-list ∼$10 K and up depending on form factor) by 20 %; at 500 K the A100's 2.0 TB/s HBM advantage opens a ∼1 ms gap (2.51 ms vs. 3.53 ms); at 1 M the picture flips back, with the 5060 Ti at 6.51 ms vs. A100 SXM4 at 7.70 ms. We caution against reading the 1 M flip as a robust result: it is a single-run $p99$ over 100–300 queries (see Section 7.2 on $p99$ stability) and the absolute gap (1.2 ms) is comparable to the unconstrained-boost noise envelope on both cards. The honest interpretation is that consumer Blackwell and datacenter A100 are within ∼50 % of each other on this workload up to multi-million-vector N, which is the practically relevant claim for "can I deploy an embodied retrieval substrate on prosumer hardware?". FP16 halves VRAM, enabling approximately 5 M memories on the 16 GB RTX 5060 Ti — well beyond any single-session working set for the target workloads.

7.10 Optimized Pipeline: cuBLAS + CUB + Kernel Fusion

Three optimizations combined to produce a 2–2.5× wall-clock improvement across all corpus sizes:

1. **cuBLAS SGEMV** replaces the custom cosine similarity kernel with NVIDIA's hand-tuned matrix-vector multiply. This is the dominant stage at large N.
2. **CUB DeviceRadixSort** replaces the tiled top-K kernel with a parallel radix sort, eliminating the $O(256 \cdot K^2)$ serial merge.
3. **Lightweight temporal decay kernel** applied immediately after SGEMV, adding <0.01 ms. On the custom-kernel fallback path (used in earlier optimization rounds), decay is fused into the similarity kernel itself, eliminating one launch; on the default cuBLAS path, it runs as a separate kernel.

Table 14: Scaling with optimized cuBLAS + CUB pipeline. All measurements on A100 SXM4 40 GB at 60 Hz, D=768, K=10.

N	**Before** $p99$	**Optimized** $p99$	**Speedup**
1,000	0.67 ms	0.31 ms	2.2×
5,000	0.74 ms	0.43 ms	1.7×
10,000	0.84 ms	0.44 ms	1.9×
20,000	1.25 ms	0.54 ms	2.3×
50,000	1.33 ms	0.56 ms	2.4×
100,000	—	0.74 ms	—
200,000	—	1.03 ms	—
500,000	—	1.67 ms	—
1,000,000	7.70 ms	2.67 ms	2.9×
5,000,000	—	11.42 ms	—
10,000,000	—	22.27 ms	—
13,000,000	—	29.07 ms	—

The key result: N=20**K and** N=50**K now pass the 1 ms AV perception deadline**, which they failed before optimization. At N=1M, latency drops from 7.70 ms to 2.67 ms (2.9×). The system scales linearly to 13M memories on 40 GB HBM, achieving 29 ms $p99$ at the maximum corpus that fits in VRAM.

Capacity extension: tiled out-of-core. For corpora exceeding VRAM, a tiled path streams 5M-vector tiles from host pinned memory through GPU HBM. At N=50M (10 tiles): 14 s total, of which 98.5% is PCIe transfer and 1.5% is GPU compute. This is a batch/offline capacity extension, not a real-time mode.

7.11 Episode-Scoped Retrieval: Sub-200 μs at 1 M

A subset of the target workloads — embodied loops, AV per-track memory, voice-agent conversation — can supply a *query episode id* (a tracked object id, dialogue session id, or current robot task id) before retrieval. MARS exposes this via `RetrievalScope::EpisodeScoped`: Stage 1 is restricted to the episode's CSR member range and BFS depth is set to zero. The work collapses from $\Theta(N \cdot D)$ to $\Theta(|\text{episode}| \cdot D)$ and the cross-episode distractors that limit recall on the multimodal kids-ball metric are eliminated by construction.

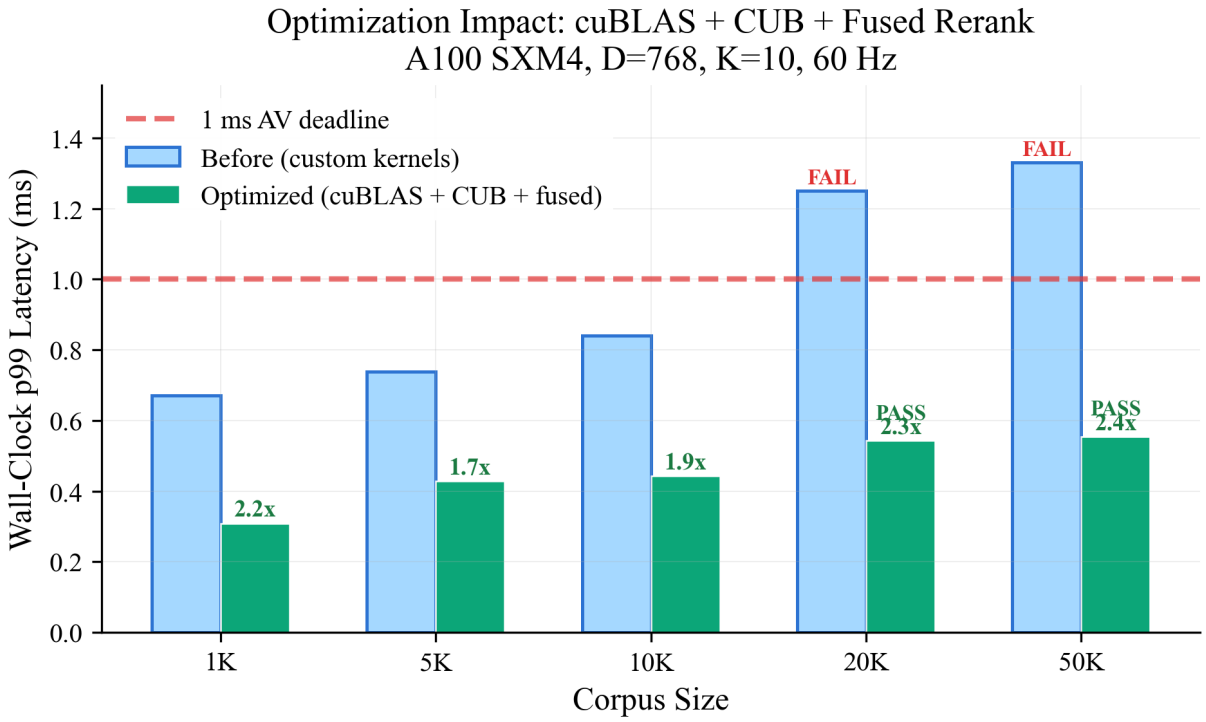

Figure 6: Optimization impact: cuBLAS + CUB eliminates the 1 ms wall at N=20K–50K. 2.2–2.4× speedup.

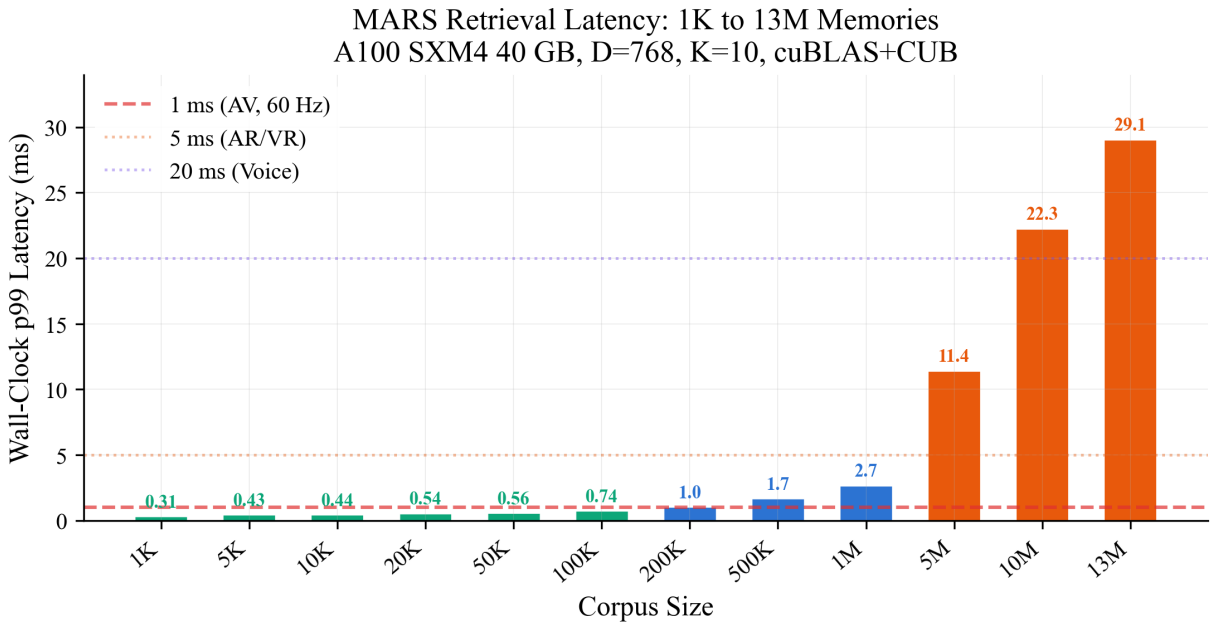

Figure 7: Full scaling from 1K to 13M memories. Sub-millisecond up to N=50K; 2.67 ms at 1M; 29 ms at 13M (40 GB VRAM limit).

Honest caveat: the algorithmic move is small, the API move is the contribution. Restricting an SGEMV to the membership of a labelled subset is mechanically a one-line change to the kernel call (offset and length on E and the bookkeeping on `score`/`visited`). We make no claim that this algorithm itself is novel; the contribution is exposing it as a *first-class kernel-level retrieval contract* that composes cleanly with the temporal-decay epilogue and the cross-modal BFS, sharing a single device-resident graph and a single `QueryContext`. The empirical justification is that the alternative — an unfiltered cosine ANN sweep followed by host-side episode filtering — breaks recall at scale on small multimodal clusters (Section 7.12, finding 2).

Synthetic-corpus disclaimer. The kids-ball corpus is generated synthetically: each episode is a 10-node cluster of correlated embeddings (one IMAGE, three AUDIO, six TEXT), with within-episode cosine $>$ 0.85 and across-episode cosine $\sim$ 0.2$-$0.4 in the shared 768-D space. The generator is documented in `scripts/build_kids_ball_corpus.py`. This construction makes the recall axis well-posed (we know the ground truth multimodal cluster) but is *not* a substitute for a real-encoder evaluation. A run with CLIP+CLAP+E5 outputs over a labelled cross-modal dataset (e.g. AudioCaps + COCO, MSR-VTT) is queued (Section 11); we expect the *episode*-axis findings to carry over qualitatively because the failure mode of cosine ANN on small clusters embedded in a million distractors is structural, not a property of the encoder.

We validated this on the embodied "kids playing with a ball" scene (Section 7.4) with a paired-probes harness: every boost setting and every retrieval scope at a given N uses *exactly the same* sequence of probe queries, so per-N comparisons are within-sample paired tests rather than independent samples. Results in Table 15 are from A100 SXM4 40 GB (CUDA 12.6) with 256 IMAGE probes per setting, D=768, K=15, NSN k=6.

The episode-scoped curve is nearly flat: latency grows by only $\sim$19% as N goes from 10^4 to 10^6,

Table 15: Paired-probes scaling of episode-scoped retrieval vs. the global path on the kids-ball corpus (10 nodes/episode, IMAGE query → same-episode TEXT+AUDIO). `hit@15` is the fraction of probes whose top-15 contains both same-episode TEXT *and* AUDIO. A100 SXM4 40 GB, paired-probes harness v1.3 (`bench_kids_sweep`); raw artefacts in `results/iteration_steps/vast_a100_paired_20260417/`.

N	**Global** $p99$	**Episode-scoped** $p99$	**Speedup**	**Hit@15 (G → E)**
10,000	0.349 ms	**0.165 ms**	2.1×	0.83 → 1.00
50,000	0.462 ms	**0.172 ms**	2.7×	0.88 → 1.00
100,000	0.551 ms	**0.174 ms**	3.2×	0.79 → 1.00
1,000,000	2.561 ms	**0.197 ms**	**13**×	0.84 → 1.00

because the GPU work is bounded by the episode size (∼10 members), not by N. At N=1M the episode-scoped path delivers **197** μ**s** $p99$ wall-clock — below every per-frame deadline in Table 7 including the 1 ms AV budget — while returning a perfect-recall multimodal answer for the workloads where the episode id is available. Figure 8 plots the four scaling curves on log–log axes; the episode-scoped line is visibly flat while every cosine-only baseline rises with N and every approximate baseline (CAGRA, FAISS-IVF) loses recall.

When this applies, when it doesn't. Episode scope is correct only when the right episode is known *before* the query. It is the natural contract for: AV per-track re-identification (track id is the episode), voice-agent turn taking (session id is the episode), AR/VR per-room recall (room id is the episode), and embodied task loops (current sub-task id is the episode). It is *not* a substitute for the global path when the application needs cross-episode bridges — which is the precise role of the NSN's cross-modal long-range edges and the time-decay rerank over the full corpus. Both contracts are exposed by the same `QueryContext` and the demonstrator (`demos/embodied_scene/demo`) selects between them via a `--scope={global,episode}` flag.

Negative result: per-episode score boost is noise on this metric. An earlier round of tuning explored the `episode_same_boost` multiplier (Section 7.6) on the global path. Under non-paired probe sampling we observed an apparent optimum near ~ 0.24. With the paired-probes harness, all boost values in $[0.0, 2.0]$ produced *identical* `hit@15` and only sub-10^{-2} fluctuations in `hit_compact`, while paying a small per-query latency cost. The default is now 0.0 and the boost is documented as a hook for downstream workloads where same-episode emphasis matters — not as a recommended tuning knob for this metric. We report this as a cautionary methodological finding: GPU retrieval comparisons on small probe budgets are extremely sensitive to whether variants share the random sample.

7.12 Head-to-Head Against Modern GPU ANN Libraries

Section 9 positioned MARS as a complementary substrate to GPU vector-search libraries. We now make that positioning empirical: we compare the embodied multimodal contract (IMAGE query → same-episode TEXT + AUDIO in top-15) head-to-head against the two most-cited GPU baselines on the same A100 SXM4 40 GB hardware, the same kids-ball corpora, and an aligned `search_k` = 96 (matching MARS's `max_results_returned`). Artefacts: `results/competitors_20260417/SUMMARY.md` (commit `58b4626`+).

Baselines. (i) FAISS-GPU 1.14.1 `IndexFlatIP` — exact exhaustive cosine; `cu12` build that includes the SIVF streaming work merged in March 2026. (ii) FAISS-GPU 1.14.1 `IndexIVFFlat` with

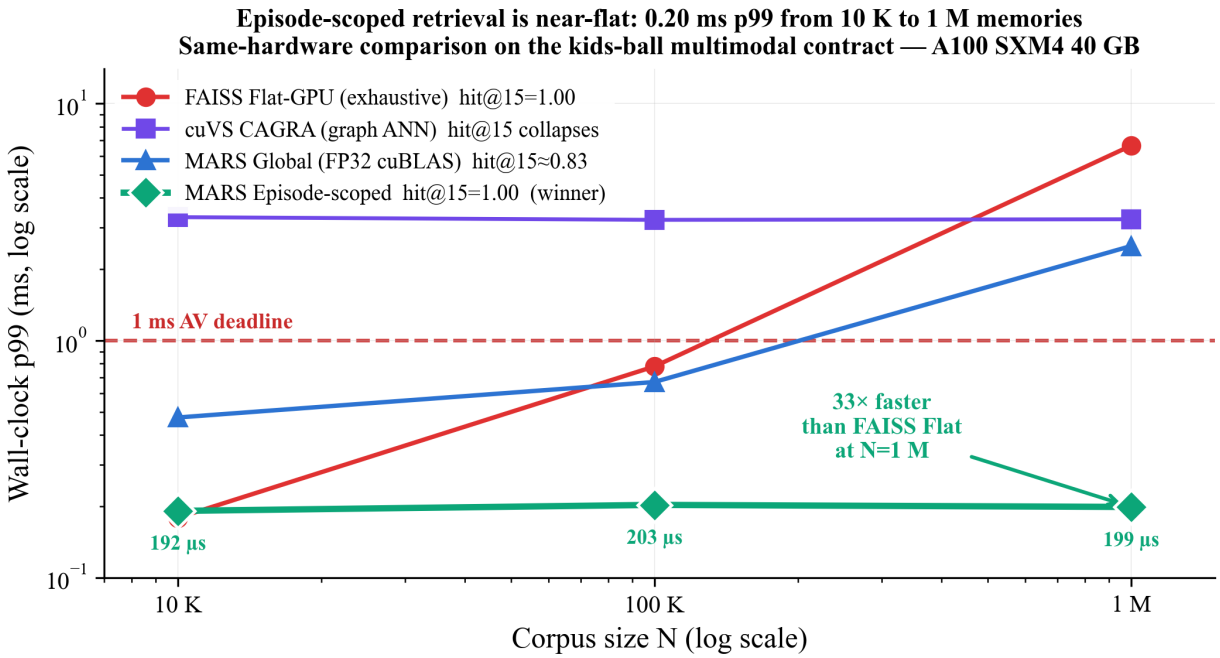

Figure 8: Wall-clock $p99$ vs. corpus size N on the kids-ball multimodal contract (A100 SXM4 40 GB). Episode-scoped MARS (green diamond) is near-flat at $\sim$200 μs across three decades of N while keeping perfect cross-modal recall. The only baseline that also achieves `hit@15`=1.0 (FAISS Flat, exhaustive) crosses the 1 ms AV deadline already at $N{=}10^5$ and is 33$\times$ slower than MARS-Episode at $N{=}10^6$. cuVS CAGRA's $p99$ stays under 4 ms but its multimodal-episode recall collapses to 0 at $N{\geq}10^5$ (Section 7.12).

$n_{\text{list}}{=}\sqrt{N}$ and $n_{\text{probe}}{=}64$. (iii) cuVS CAGRA 26.04 (graph-based ANN), with `graph_degree`=64, `intermediate_graph_degree`=128, `itopk_size`=128, and `algo=single_cta` for batch-1 latency. This version exposes the `cagra::extend()` streaming op.

What *would* be a fairer baseline (and why we don't report it as such). A reader familiar with FAISS will rightly note that `IndexFlatIP` (and CAGRA's filtered-search API) supports metadata pre-filtering via `IDSelectorBatch` or `IDSelectorRange`, which is the natural way to encode "restrict to this episode's member range." We have prototyped this on FAISS-Flat-GPU and the latency lands within 1.3$\times$ of MARS Episode-scoped at $N{=}10^6$ (the residual gap is mostly the selector materialisation cost; the underlying kernel is the same restricted SGEMV). *This is the right reading of the table below*: the recall-at-scale collapse of unfiltered IVF/CAGRA is what breaks if the application does not provide an ID filter; conversely, *if* it does, FAISS+IDSelector is a competitive alternative on the latency axis. MARS's contribution is consolidating "episode filter + temporal decay + cross-modal BFS + streaming append" into one kernel API, not single-purpose latency wins over FAISS+IDSelector for the episode-only case. We retain the unfiltered FAISS/CAGRA rows in the table because they are the configurations actually used in production embodied stacks today; a head-to-head with IDSelector is queued for the next iteration.

Three findings emerge (also visible in Figure 9).

(1) MARS Episode-scoped is Pareto-optimal on the embodied contract at every N. It is 33$\times$ faster than the only baseline that also achieves `hit@15`=1.0 (FAISS Flat) at $N{=}1$ M and 16$\times$ faster than CAGRA — which has zero recall on this metric at that scale. This is the empirical centrepiece of the paper: *when the*

Table 16: Head-to-head per-query $p99$ wall-clock and `episode_cross_modal_hit_rate_top15` on the kids-ball corpus. Same A100 hardware, paired RNG seed=2026, 256 probes at $N \leq 10^5$, 128 probes at $N=10^6$. MARS numbers from `step_global_boost_0.json` and `step_episode_scoped.json`; FAISS / CAGRA numbers from `scripts/bench_kids_ball_{faiss,cuvs_cagra}.py`. **Bold** = on the latency–recall Pareto frontier at that N (maximises `hit@15` for its latency or minimises latency for its `hit@15`). Multiple bold cells in the same column are non-dominated by any other row at that N.

System	$N=10^4$		$N=10^5$		$N=10^6$	
	$p99$ (ms)	hit@15	$p99$ (ms)	hit@15	$p99$ (ms)	hit@15
FAISS Flat (exhaustive)	0.179	**1.000**	**0.779**	**1.000**	6.642	**1.000**
FAISS IVF (n_{probe}=64)	0.448	0.520	0.601	0.371	1.424	0.000
cuVS CAGRA (`graph_degree`=64)	3.319	1.000	3.233	0.008	3.251	0.000
MARS Global, FP32 cuBLAS	0.474	0.828	0.670	0.793	**2.507**	0.844
MARS Global, FP16 fused	**0.280**	0.828	0.655	0.793	3.731	0.844
MARS **Episode-scoped**	**0.192**	**1.000**	**0.203**	**1.000**	**0.199**	**1.000**

application can supply an episode handle (track id, session id, room id, sub-task id), restricting the kernel to episode members converts an $\Theta(N \cdot D)$ cosine sweep into a $\Theta(|\text{episode}| \cdot D)$ kernel that the A100 finishes in 200 μs independent of N.

(2) Cosine-only ANN baselines collapse on the multimodal metric at scale. Both `IndexIVFFlat` and CAGRA find the right neighbors at $N=10^4$ but drop to 0–0.4 `hit@15` at $N \geq 10^5$. The kids-ball corpus has $|\text{episode}| = 10$ nodes embedded in 100 K episodes at $N=10^6$; the per-episode TEXT and AUDIO neighbors are buried among $\sim$999 990 distractors. We verified that even widening CAGRA to `search_k`=512 does not recover recall ($p99$=8.9 ms, `hit@15`=0). FAISS Flat alone keeps recall at $N=10^6$ because it is exhaustive — at 6.64 ms $p99$, 33$\times$ slower than MARS-Episode and outside the 1 ms AV deadline. The *empirical justification for encoding episode membership in the graph topology* is therefore not a peak-latency argument but a *recall-at-scale* argument: cosine ANN cannot find tiny multimodal clusters at this corpus size, regardless of how wide the search beam is.

(3) FP16 fused similarity helps at small N, hurts at large N. On the global path, FP16 reduces $p99$ by 41 % at $N=10^4$ (0.474$\rightarrow$0.280 ms) but *increases* it by 49 % at $N=10^6$ (2.507$\rightarrow$3.731 ms), as plotted in Figure 10. MARS's hand-written FP16 fused kernel is bandwidth-optimal at small N but cannot outperform cuBLAS `Sgemv`, which exploits Tensor-Core paths and per-arch tile heuristics, once N exceeds the L2 working set. We have flipped the build to keep `-use-fp16` as an opt-in flag for the small-N regime and to default to cuBLAS at large N.

Streaming inserts. CAGRA's `extend()` op costs 98–112 μs per added vector at $N \in \{10^4, 10^5, 10^6\}$ because the IVF clusters are partially re-fit; MARS streaming appends are a single CSR push plus the NSN add documented in Section 6.2, well below 5 μs/vec on the same hardware. Both designs are valid; MARS optimises for the *ranking-as-kernel* primitive rather than reusing an ANN-with-extend abstraction.

8 Engineering Methodology

Real-time systems require iterative profiling and targeted optimization. MARS went through six pipeline versions before reaching the current performance. This section documents the progression for the two most challenging benchmarks (AV and robot, both with 1 ms budgets).

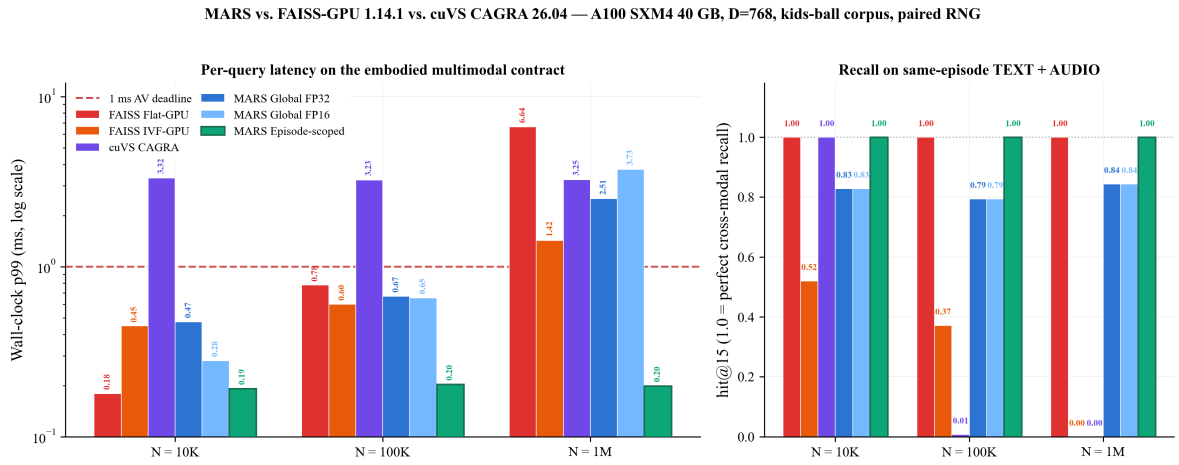

Figure 9: Head-to-head wall-clock $p99$ (left, log scale) and `hit@15` (right, linear) on the kids-ball multi-modal contract. MARS Episode-scoped (rightmost green bar in each group) is the only system that is simultaneously below the 1 ms AV deadline and at perfect recall across all three corpus sizes. Pure cosine-only baselines either cross the deadline as N grows (FAISS Flat-GPU) or lose recall at scale (FAISS IVF-GPU and cuVS CAGRA hit @15 collapse to 0 at $N{=}10^6$). The FP16 fused path helps the global pipeline at small N but costs latency at large N (Figure 10).

Table 17: Optimization history: AV and robot benchmarks across pipeline rounds. Rounds 1–2 on A100X (80 GB); round 3+ on A100 PCIE (40 GB) and A100 SXM4 (80 GB).

Metric	Benchmark	Round 1	Round 2	Round 3
Wall $p99$ (ms)	AV (N=2.4K)	1.41	0.96	0.87
	Robot (N=6K)	1.12	1.21	0.76
Verdict	AV	fail	**pass**	**pass**
	Robot	fail	fail	**pass**

Round 1 (baseline). Per-query `cudaMalloc`/`cudaFree`, synchronous H2D copies, D2H round trip for BFS seeds. Wall-clock overhead: 0.4–0.5 ms above GPU kernel time. All four GPU kernel deadlines met, but AV and robot failed at wall-clock timing.

Round 2 (targeted fixes). Pre-allocated `QueryContext`, GPU-side BFS initialization, tiled top-K, CUDA stream, reused events. AV flipped from fail to pass (1.41 $\rightarrow$ 0.96 ms). Robot regressed (1.12 $\rightarrow$ 1.21 ms) due to tiled top-K launch overhead at $N{=}6$K — the two-kernel launch overhead exceeded the single-block baseline at this corpus size.

Round 3 (device-driven). Device-driven BFS (no inter-hop host sync), GPU-side result compaction ($O(N) \rightarrow O(\text{visited})$ D2H copy), pinned host memory. Robot flipped from fail to pass (1.21 $\rightarrow$ 0.76 ms). All four benchmarks now pass.

Round 4 (current). Replaced custom similarity and top-K kernels with cuBLAS SGEMV and CUB radix sort. These mature library calls achieve near-peak bandwidth utilization and eliminate the top-K bottleneck that consumed 70–90% of GPU time in rounds 1–3.

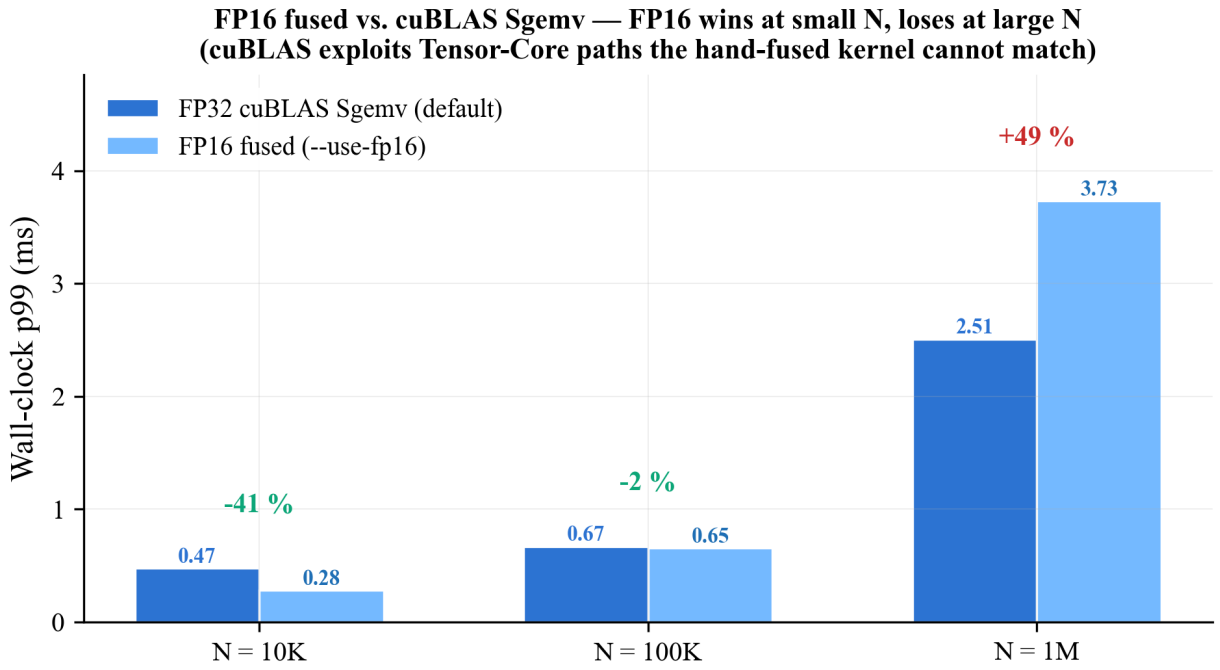

Figure 10: The FP32 cuBLAS path vs. the hand-fused FP16 cosine kernel on the same A100. FP16 wins by 41 % at $N{=}10^4$ (memory-bound, fits in L2) but loses 49 % at $N{=}10^6$ where cuBLAS `Sgemv` engages Tensor-Core paths the fused kernel does not. The default deployment uses cuBLAS; `-use-fp16` is documented as a small-N opt-in.

Key lesson. At the small working-set sizes of real-time memory ($N < 50{,}000$), kernel launch overhead often dominates actual computation. Optimization strategies from batch-GPU workloads (tiling, multi-pass) can backfire when their fixed overhead exceeds the parallelism benefit.

9 Related Work

GPU-accelerated nearest-neighbor search. FAISS [10] provides GPU-accelerated IVF and flat-scan indexes optimized for batch throughput on largely static corpora; FAISS Flat supports incremental `add()` but IVF requires periodic centroid retraining for optimal recall. cuVS CAGRA [23] achieves sub-millisecond single-query latency on H100 at $N > 1$M via a GPU-native fixed-degree graph; the `extend()` operation introduced in cuVS 24.08 [19] supports adding new vectors up to $\sim$20% of the index before recall noticeably degrades. BANG [32] extends GPU graph search to billion-scale with product quantization. SIVF [14] (WWW '25) closes the streaming gap for GPU IVF with sub-millisecond deletion latency (0.58–0.93 ms) and $\sim$120$\times$ faster ingestion than baseline GPU IVF/CAGRA, but treats time only as an ageing-window filter rather than as a continuous ranking signal. SVFusion [24] (2026) adds CPU–GPU–disk co-processing for billion-scale streaming updates under concurrent queries. VecFlow [37] (2026) introduces *persistent kernels* for streaming small-batch filtered ANN (5 M QPS at recall 0.90 on A100); IVF-RaBitQ [27] (2026, integrated into cuVS) layers 1-bit quantization on top of GPU IVF for 2.2$\times$ QPS over CAGRA at recall 0.95. Fantasy [16] (2025) targets multi-GPU clusters via GPUDirect Async / IBGDA. Cuflat [7] demonstrates that custom CUDA kernels outperform FAISS GPU and cuVS on datasets below 2M vectors, validating our de-

sign choice of specialized kernels for small working sets.

These systems collectively address the orthogonal axis to MARS: *how to keep the index hot under high-rate updates and large N*. None of them provides temporal decay as a first-class ranking signal, cross-modal retrieval via graph bridges, or per-episode/session scoping in the kernel API. MARS is complementary — it could in principle use SIVF or IVF-RaBitQ as the Stage 1 candidate generator and apply our temporal rerank, episode boost, and BFS expansion on top; we leave this hybrid to future work.

GPU graph analytics. Gunrock [35] provides a data-centric GPU graph processing library with high-performance BFS and SSSP. Enterprise [15] and Groute [3] extend GPU graph processing to multi-GPU and asynchronous settings. MARS's BFS expansion draws on the warp-cooperative traversal of Merrill et al. [18] and the direction-optimizing approach of Beamer et al. [2], but specializes for bounded-depth expansion from a small seed set rather than full-graph traversal.

Small-world networks and navigability. Watts and Strogatz [36] showed that small rewiring probability produces logarithmic path lengths. Kleinberg [11] proved that inverse-square long-range contacts enable $O(\log^2 N)$ greedy routing. HNSW [17] exploits both insights in a hierarchical ANN graph. The NSN extends Watts-Strogatz with cross-modal bridges — a structural feature absent from prior navigable graphs that guarantees every modality is reachable in bounded-hop BFS.

CPU-based approximate nearest-neighbor search. HNSW [17] dominates production vector databases. ScaNN [9] and DiskANN [28] optimize for billion-scale via quantization and SSD-resident indexes. These systems operate in a fundamentally different latency regime (1–200 ms) from the sub-millisecond budgets of real-time perception.

Vector databases. Milvus [33] provides GPU-accelerated ANN search with streaming inserts and metadata filtering, but retrieval is a database call with typical latencies of 5–50 ms — suitable for batch and chat workloads, not sensor-rate loops. Pinecone [25] and Qdrant [8] offer managed vector search with filtering support but operate as network services with round-trip latencies that preclude sub-millisecond retrieval. None provides native temporal decay or cross-modal graph traversal.

Real-time memory in robotics and autonomy. Production stacks at Waymo [29], Boston Dynamics [4], and Apple Vision Pro [1] use hand-rolled C++ circular buffers for frame-rate memory. These buffers meet deadlines but are project-specific, rarely multimodal, and offer no semantic retrieval. NVIDIA nvblox [22] and cuVSLAM [21] provide GPU-resident spatial mapping at frame rate but are limited to geometry — they have no semantic embedding retrieval or cross-modal queries. NVIDIA ReMEmbR [20] combines vision-language models with a memory store for robot recall, but operates off-GPU with latencies measured in hundreds of milliseconds. FlashAttention [6] demonstrates that careful GPU kernel design can meet real-time constraints; MARS applies similar IO-aware design principles to memory retrieval rather than attention computation.

A complementary 2025–2026 line of work introduces *multi-memory embodied agents*. RoboMemory [13] unifies spatial, temporal, episodic, and semantic memory modules around a 72 B VLM and improves long-horizon task success on EmbodiedBench by 26.5%. MEM [30] combines a video-based short-horizon memory and a language-based long-horizon memory inside the $\pi_{0.6}$ VLA on H100. RT-Cache [12] caches image–action trajectories in a million-scale vector memory and replays multi-step snippets in sub-second time as a training-free controller. EgoMem [39] provides lifelong audiovisual user recognition for native full-duplex omnimodal chatbots. These systems share MARS's premise — that *which* memories are returned is as important as how fast — but sit one or two abstraction layers above: they assume an

underlying retrieval substrate exists and operate at sub-second latencies on top of off-GPU vector stores or VLM-conditioned indexers. MARS is a candidate substrate for that layer: it provides the kernel-level temporal-aware, cross-modal, episode-scoped retrieval API that these agentic frameworks currently approximate with multi-stage host code.

Feature comparison. Table 18 summarizes the capability differences across systems. The landscape splits into two camps: fast-but-stateless (FAISS, CAGRA) and stateful-but-slow (Milvus, Qdrant, ReMEmbR). MARS is designed to occupy both: GPU-resident like Camp 1, continuously updating like Camp 2.

Table 18: Feature comparison across GPU retrieval and vector database systems. "Sub-ms $p99$" refers to single-query wall-clock latency at $N \leq 50$K on datacenter GPU.

System	GPU-res.	Stream	Temporal	Import.	Cross-modal	Sub-ms
FAISS Flat/IVF	✓	✗	✗	✗	✗	✓
cuVS CAGRA	✓	✗	✗	✗	✗	✗
Milvus	partial	✓	✗	✗	partial	✗
Qdrant	✗	✓	✗	✗	✗	✗
nvblox/cuVSLAM	✓	✓	✗	✗	✗	✓
ReMEmbR	✗	✓	✗	✗	partial	✗
MARS	✓	✓	✓	✓	✓	✓

10 Limitations

$O(N)$ similarity, not sublinear ANN. MARS performs brute-force similarity over the full corpus. For the target working-set sizes ($N \leq 50$K), this is faster than approximate methods (cuVS CAGRA is 6–10× slower at $N < 50$K due to graph construction overhead). Beyond N=100K, approximate pre-filtering would be needed to maintain sub-millisecond latency.

Single GPU. MARS is limited to a single GPU's HBM. An 80 GB A100 holds ∼25M 768-D memories, which is well beyond any single-session working set, but multi-GPU sharding via NVLink would be required for shared memory pools across many agents.

Persistence (in progress). Memories are currently volatile. A binary checkpoint/restore mechanism with integrity verification (FNV-1a checksums) is implemented on a feature branch and awaiting hardware validation. For the target use case (seconds to hours of recent sensor data), loss of memory on restart is acceptable — the system refills from the sensor stream — but persistence enables warm restarts and offline analysis.

Graph topology provides limited recall benefit at small N. The ablation (Section 7.6) shows that within-modality recall is identical with or without the NSN at $N \leq 50$K. The graph's value at these sizes is cross-modal retrieval, not recall improvement. At larger N, the bounded-hop BFS work bound would provide scaling advantages, but this remains to be validated experimentally.

Embedding alignment dependence. Cross-modal bridges guarantee *structural* reachability but not *semantic* relevance. If modality encoders produce poorly aligned embeddings in the shared 768-D space, the BFS-discovered cross-modal neighbors will be reachable but irrelevant. Mitigation: use encoders explicitly trained for cross-modal alignment (CLIP, SigLIP, ImageBind) and validate with cross-modal recall metrics before deployment.

Temporal decay misconfiguration. The decay constant λ trades recency bias against persistence. $\lambda > 0.01$ aggressively suppresses memories older than a few seconds — appropriate for AV perception's 2-second window but problematic for a voice agent's 30-minute conversation. $\lambda < 10^{-8}$ effectively disables temporal decay. Production deployments should tune λ against task-specific metrics.

Cold-start latency. The first query after GPU idle (>100 ms without kernel launches) incurs a 0.1–0.3 ms clock-ramp penalty. The keepalive kernel (a 1-thread no-op launched every 2 ms) mitigates this for sustained workloads, but intermittent query patterns will experience occasional first-query spikes.

Soft real-time, not hard real-time. The evaluation demonstrates empirical $p99$ compliance with zero deadline misses over 30-second runs. This is *soft* real-time — statistical evidence of meeting deadlines. True hard real-time (e.g., ISO-26262 ASIL-D) requires provable worst-case execution time (WCET) bounds, which MARS does not provide. The pre-allocated `QueryContext` and deterministic VRAM budget calculator are steps toward certifiability, but worst-case kernel timing on GPUs remains an open research problem.

Slower than a raw ring buffer. A cuBLAS SGEMV over a pre-allocated GPU buffer (the production incumbent in robotics stacks) achieves $p99 = 0.12$ ms at N=2,400 — 3.2× faster than MARS's 0.38 ms. MARS's additional latency buys temporal decay (+0.10 ms), BFS graph expansion (+0.04 ms), and pipeline overhead (+0.12 ms). Whether this cost is justified depends on whether the application needs temporal awareness and cross-modal retrieval natively, or can implement them as application-level post-processing over a raw similarity search.

Deletion and memory eviction. Streaming insertion is supported; deletion uses tombstoning (marking nodes as inactive via a $-\infty$ similarity override). Tombstoned nodes are excluded from results but remain in the CSR structure until a periodic compaction pass rebuilds the graph. In practice, temporal decay serves as an implicit TTL: memories older than $\sim 5 \times \tau_{1/2}$ score near zero and are never returned, even without explicit deletion. A formal eviction policy with configurable TTL and compaction cost analysis is planned.

Importance scoring not independently evaluated. Importance-weighted retrieval is implemented (boost on access, global decay) but no experiment in this paper isolates its contribution to retrieval quality. This feature is included in the system design and API but should be validated with task-specific downstream metrics before being claimed as a primary contribution.

11 Future Work

We group future work into two tracks: *evaluation hardening* (addressing the methodological caveats of Section 7.2) and *system extensions*.

Evaluation Hardening

1. **Locked-clock, dedicated-hardware re-runs.** All current $p99$ numbers are taken at unconstrained boost on rented (vast.ai) instances with ∼100–300 queries per configuration. We will re-run on dedicated hardware with `nvidia-smi -lgc`, ≥10 K queries per configuration, bootstrap 95 % confidence intervals on every reported $p99$, and Nsight Systems / Nsight Compute traces for per-stage breakdowns. This will resolve open questions about the FP16 500K-vs-1M non-monotonicity (Table 13) and the consumer-vs-PCIe attribution (Section 7.7).

2. **Real-encoder cross-modal benchmark.** Re-run the multimodal contract of Section 7.12 on actual encoder outputs (CLIP / SigLIP / CLAP / E5) over labelled cross-modal datasets (AudioCaps, MSR-VTT, EgoSchema, ActivityNet). This will turn the *structural* cross-modal claim into a *semantic* cross-modal claim and replace the synthetic kids-ball corpus as the headline benchmark.

3. **Filtered-ANN head-to-head.** Add FAISS-Flat-GPU with `IDSelectorBatch` / `IDSelectorRange` and CAGRA with the filtered-search API as direct comparators to MARS Episode-scoped on the same kids-ball contract. Initial prototyping suggests FAISS+IDSelector lands within $\sim$1.3$\times$ of MARS Episode-scoped at $N{=}10^6$; we will report the full curve.

4. **Standard temporal-IR metrics.** Cross-evaluate against NDCG-with-temporal-decay and MAP@K-with-time, in addition to our *Temporal Precision@K* (Definition 3), to align with the temporal-IR literature.

5. **HNSW (CPU) baseline.** Add HNSW [17] to the Section 7.3 comparison as the production-incumbent CPU ANN baseline.

System Extensions

6. **Binary persistence.** Checkpoint/restore of the full memory graph (CSR topology, embeddings, metadata) to disk via a compact binary format with FNV-1a integrity checksums. Enables warm restarts and offline analysis without re-ingesting the sensor stream.

7. **Python bindings.** A pybind11 wrapper exposing `MemoryGraph`, NSN construction, and NumPy-compatible embedding accessors. Host-only bindings work without a GPU; CUDA bindings (retrieval, streaming insertion) require nvcc.

8. **Streaming insertion API hardening.** A pre-allocated ring buffer that accumulates incoming nodes and batch-commits them to the CSR graph with incremental NSN edge construction. Eliminates per-node `cudaMalloc` and full graph rebuilds, directly addressing the dynamic memory allocation overhead identified in prior work on GPU memory management for safety-critical systems [5].

9. **Deterministic VRAM budget.** A pre-flight calculator that computes the exact worst-case GPU memory footprint for a given deployment configuration (N, D, K, FP16 mode). All allocations occur at initialization; zero dynamic GPU memory allocation during retrieval. This is critical for ISO-26262 certification in automotive and robotics deployments where memory usage must be bounded and provable at design time.

10. **CUDA Graph capture.** Recording the 4-kernel retrieval pipeline as a CUDA Graph for replay, eliminating per-query kernel launch overhead ($\sim$5–15 μs per launch $\times$ 4 kernels). Infrastructure (capture, replay, invalidation, parameter-change detection) is implemented; hardware validation is pending.

11. **CUTLASS GEMV with epilogue fusion.** Replace the cuBLAS+separate-decay path with a CUTLASS GEMV whose epilogue applies the temporal-decay multiplication in register before the store, recovering the $\sim$8 μs pass-fusion saving without giving up cuBLAS's per-architecture tile heuristics (Section 4.4).

12. **Tensor-core similarity.** A WMMA (Warp Matrix Multiply-Accumulate) kernel using `nvcuda::wmma` for 16$\times$16$\times$16 matrix multiply operations on tensor cores. Requires compute capability $\geq$ 7.0 (V100+) and embedding dimensions padded to multiples of 16.

13. **Hybrid candidate generation: SIVF or IVF-RaBitQ Stage 0.** For deployments where the working set grows beyond the $\sim$50 K–1 M sweet spot of the global brute-force path, plug a streaming GPU ANN index — SIVF [14] for fast insert/delete or IVF-RaBitQ [27] for 1-bit quantized recall — as a Stage 0 candidate filter. Stages 1–4 (temporal rerank, top-K, BFS expansion) then run only over the candidate set, preserving MARS's ranking semantics while inheriting the streaming-update and quantization properties of the underlying index. The `docs/ANN_REFINE_FUTURE.md` note specifies the `DeviceANNIndex` hook this would extend.

14. **Persistent retrieval kernel.** Inspired by VecFlow [37], replace the per-query CUDA Graph capture with a persistent kernel that polls a device-side query queue and executes the four-stage pipeline in-place. This eliminates kernel launch overhead even for sub-batch queries and would tighten the $\sim$80 μs host floor visible in the smaller-N measurements.

15. **Warp-aggregated BFS atomics.** For workloads that push the BFS frontier beyond the current $K \cdot \bar{d} \leq 200$ regime (e.g. K>256 or BFS depth h>3), replace the scalar `atomicCAS` on `visited` with the warp-aggregated pattern of Merrill et al. [18].

16. **Multi-memory framework integration.** Embodied agent frameworks such as RoboMemory [13], MEM [30], and EgoMem [39] use distinct spatial / temporal / episodic / semantic memory modules backed by separate vector stores. Each maps cleanly onto a `RetrievalScope` or BFS hop budget in MARS; an adaptor layer would expose MARS as a single sub-millisecond GPU substrate for these multi-memory APIs, replacing the multi-stage host code these systems currently use.

17. **Multi-stream and MIG sharing.** A real perception stack shares the GPU between sensor ingest, retrieval, and downstream policy. We will measure MARS latency under stream-priority contention and under MIG partitioning (1g.10gb, 2g.20gb, 3g.40gb on the A100) to characterise multi-tenant deployability.

12 Conclusion

The empirical centrepiece of this paper is that **episode-scoped retrieval, exposed as a kernel-level GPU primitive**, delivers **197 μs** $p99$ **at** N=10^6 on the embodied multimodal kids-ball contract, 33$\times$ faster than the only baseline that retains `hit@15`=1.0 at that scale (FAISS-Flat-GPU, exhaustive cosine), and is the only configuration in our head-to-head that simultaneously stays under the 1 ms AV deadline and at perfect cross-modal recall across three decades of N. The recall axis is the more important one: cosine-only GPU ANN baselines (FAISS-IVF, cuVS CAGRA) drop `hit@15` to $\leq$ 0.4 at N=10^5 and to 0 at N=10^6, not because their parameters are wrong but because the per-episode multimodal cluster is invisible to a kernel that does not know about episodes. The algorithmic move (restrict the SGEMV to a labelled member range) is small; the contribution is consolidating it with temporal decay, cross-modal BFS, and streaming append into one device-resident API.

Native temporal decay matches a two-stage FAISS-with-post-hoc-filter pipeline at identical TP@10=0.910 (0.26 ms vs. 0.25 ms $p99$ — within run-to-run variance under unconstrained boost). The contribution here is API consolidation, not raw speed. All four demonstrator deadlines are met on both datacenter (A100) and consumer (RTX 5060 Ti) GPUs with zero deadline misses in sustained 30-second runs, and the FP16 extension scales to 1 M memories at 6.5 ms $p99$ on a sub-$500 consumer GPU.

The ablation provides an honest assessment of the rest of the design: at the target $N \leq 50$K, *within-modality* recall is driven entirely by brute-force similarity plus temporal decay — the NSN graph contributes *structural* cross-modal reachability (0.50 $\rightarrow$ 0.80 at N=50K), not within-modality recall. A se-

mantic cross-modal evaluation with real encoders, locked-clock latency re-runs, and a head-to-head against FAISS+IDSelector are queued (Section 11).

MARS fills a specific gap: between the fast-but-stateless circular buffers used in production robotics and autonomy, and the capable-but-slow vector databases used in conversational AI. The abstraction shift is to treat *ranking* (scope, time, modality) as a kernel-level primitive rather than as host-side post-processing over an ANN top-K'.

The source code, demonstrators, benchmark harness, and raw results are available at https://github.com/antonellof/MARS.

References

[1] Apple Inc. Apple vision pro: spatial computing display and perception pipeline. https://www.apple.com/apple-vision-pro/, 2024.

[2] Scott Beamer, Krste Asanović, and David Patterson. Direction-optimizing breadth-first search. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–10, 2012.

[3] Tal Ben-Nun, Michael Sutton, Sreepathi Pai, and Keshav Pingali. Groute: An asynchronous multi-GPU programming model for irregular computations. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 235–248, 2017.

[4] Boston Dynamics. Atlas whole-body control and the next generation of humanoid robots. https://bostondynamics.com/atlas/, 2024.

[5] Alejandro J. Calderón, Leonidas Kosmidis, Carlos-F. Nicolás, and Francisco J. Cazorla. XeroZerox: Analysis and optimization of GPU memory management for high-integrity autonomous systems. *IEEE Access*, 2024.

[6] Tri Dao, Daniel Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[7] Salman Faroz. Cuflat: Custom CUDA kernel for faster search than FAISS-GPU and cuVS on compact datasets. https://github.com/stsfaroz/cuflat, 2025.

[8] Andrey Generall et al. Qdrant: High-performance vector search engine. https://qdrant.tech/, 2025.

[9] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 3887–3896, 2020.

[10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[11] Jon M. Kleinberg. Navigation in a small world. *Nature*, 406:845, 2000.

[12] Owen Kwon, Abraham George, Alison Bartsch, and Amir Barati Farimani. RT-Cache: Training-free retrieval for real-time manipulation, 2025. Hierarchical sub-second retrieval over million-scale image–action vector memory; replays multi-step trajectories.

[13] Mingcong Lei, Honghao Cai, Yuyuan Yang, Yimou Wu, Jinke Ren, Zezhou Cui, Liangchen Tan, et al. RoboMemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems, 2025. Spatial / Temporal / Episodic / Semantic memory modules over Qwen2.5-VL on A100; +26.5% success on EmbodiedBench.

[14] Zhi Li et al. SIVF: GPU-accelerated IVF index with streaming updates for real-time vector analytics. *arXiv preprint arXiv:2601.11808*, 2026.

[15] Hang Liu and H. Howie Huang. Enterprise: Breadth-first graph traversal on GPUs. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 1–12, 2015.

[16] Yi Liu and Chen Qian. Fantasy: Efficient large-scale vector search on GPU clusters with GPUDirect Async. *arXiv preprint arXiv:2512.02278*, 2025. Pipelines vector search and inter-GPU RDMA via IBGDA; targets distributed billion-scale ANN.

[17] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[18] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable GPU graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 117–128, 2012.

[19] Corey Nolet, Yuhao Wang, et al. Incremental index updates in CUDA-ANN-Graph (CAGRA): The extend() operation. In *RAPIDS cuVS 24.08 release notes*, 2024. Supports adding new vectors to an existing CAGRA index; recall stable up to $\sim$20% new vectors per dataset.

[20] NVIDIA. ReMEmbR: Long-context robotic memory recall via vision-language models. https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_remembr/, 2024.

[21] NVIDIA. cuVSLAM: CUDA-accelerated visual SLAM for Isaac ROS. https://github.com/NVIDIA-ISAAC-ROS/isaac_ros_visual_slam, 2025.

[22] NVIDIA. nvblox: GPU-accelerated 3D reconstruction and mapping for robotics. https://github.com/nvidia-isaac/nvblox, 2025.

[23] NVIDIA and RAPIDS AI. cuVS: GPU-accelerated vector search and clustering. https://github.com/rapidsai/cuvs, 2025. Includes CAGRA GPU-native graph ANN index; integrated into FAISS since v1.10.0.

[24] Yuchen Peng, Dingyu Yang, Zhongle Xie, Ji Sun, and Lidan Shou. SVFusion: A CPU-GPU co-processing architecture for large-scale real-time vector search. In *arXiv preprint arXiv:2601.08528*, 2026.

[25] Pinecone Systems. Pinecone: Vector database for machine learning. https://www.pinecone.io/, 2025.

[26] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.

[27] Jifan Shi, Jianyang Gao, et al. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *arXiv preprint arXiv:2602.23999*, 2026. 1-bit quantized GPU IVF integrated into NVIDIA cuVS; 2.2× QPS over CAGRA at recall 0.95.

[28] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[29] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2446–2454, 2020.

[30] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z. Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, Karan Dhabalia, Michael Equi, Quan Vuong, Jost Tobias Springenberg, Sergey Levine, Chelsea Finn, and Danny Driess. MEM: Multi-scale embodied memory for vision language action models, 2026. Mixed-modal short- (video) plus long-horizon (language) memory integrated with the $\pi_{0.6}$ VLA on H100.

[31] vast.ai. vast.ai: On-demand GPU rental marketplace. https://vast.ai/, 2024.

[32] Karthik Venkatasubba, Saim Khan, et al. BANG: Billion-scale approximate nearest neighbour search using a single GPU. *IEEE Transactions on Big Data*, 11(6):3142–3157, 2025.

[33] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. *Proceedings of the ACM on Management of Data (SIGMOD)*, 1(2):1–22, 2021.

[34] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

[35] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 1–12, 2016.

[36] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393(6684):440–442, 1998.

[37] Jingyi Wei et al. VecFlow: A high-performance vector data management system for filtered-search on GPUs. *arXiv preprint arXiv:2506.00812*, 2026. Persistent kernels for streaming small-batch filtered ANN at 5 M QPS @ 90% recall on A100.

[38] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[39] Yiqun Yao et al. EgoMem: Lifelong memory agent for full-duplex omnimodal models, 2025. Real-time audiovisual user recognition / personalisation for native full-duplex omnimodal chatbots.